

EMP

# Employee Management Portal

The Simple Guide — every file, in plain English

Written for someone who is new to Angular.

Nothing is assumed. Every word is explained the first time it appears.

Angular 22 · TypeScript · 136 files · ~11,900 lines

## 0. How to read this document

---

This guide explains the whole project as if you had never written a line of Angular. It is deliberately slow. If you already know a word, skip that box — but every word *is* explained somewhere, and nothing later depends on knowledge you were never given.

Read it in this order:

1. **Part 1** is a dictionary. Read it once, quickly. Do not try to memorise it — just get familiar with the words so they are not strangers later.
2. **Part 2** shows the whole app from above: what it does and how the folders are arranged.
3. **Part 3** follows one single button click all the way through the app and back. This is the most valuable page in the document. If you understand Part 3, everything else is detail.
4. **Parts 4–9** go file by file. Each file gets: what it is, why it exists, and a table explaining its important lines.

**Green boxes** translate technical wording into everyday language.

**Amber boxes** are warnings, gotchas, or "this looks strange but here is why".

**Grey boxes** define a new word the moment it first appears.

## Contents

### 1. The dictionary

- 1.1 Basic web words
- 1.2 TypeScript words
- 1.3 Angular words
- 1.4 Signals & observables
- 1.5 HTTP words

### 2. The big picture

### 3. One click, start to finish

### 4. The starting files

### 5. core/ — the brain

- 5.1 models — the shapes
- 5.2 tokens — named settings
- 5.3 services — the workers
- 5.4 state — the memory
- 5.5 interceptors — the checkpoints

### 5.6 guards & resolvers

### 6. shared/ — reusable parts

- 6.1 components
- 6.2 directives
- 6.3 pipes

### 7. features/ — the screens

- 7.1 Login
- 7.2 Dashboard
- 7.3 Employees
- 7.4 Departments
- 7.5 Attendance & Leave
- 7.6 Admin area
- 7.7 Settings & error pages

### 8. Testing files

### 9. Configuration files

### 10. Recap tables

# 1. The dictionary

---

Every word this project uses, explained once, properly. Read it top to bottom. It will feel like a lot — but each entry is three sentences, and you will meet all of them again later.

## 1.1 Basic web words

**HTML** — the language that describes *what is on a page*: a heading, a paragraph, a button. It is a list of nested tags like `<button>Save</button>`. Think of it as the skeleton.

**CSS** — the language that describes *how the page looks*: colours, spacing, size, fonts. Think of it as the clothes on the skeleton.

**JavaScript** — the language that describes *what the page does* when you interact with it: what happens when you click, type, or scroll. Think of it as the muscles.

**DOM** (Document Object Model) — the live, in-memory version of the page that the browser actually keeps while the page is open. When we say "put it in the DOM", we mean "make it really appear on screen". When we say "removed from the DOM", the element does not exist at all — it is not merely hidden.

**Browser** — Chrome, Safari, Firefox. It downloads your files, builds the DOM, and runs your JavaScript.

**Server** — another computer, somewhere else, that sends the browser its files and data. In this project the "server" is faked (explained later), but the code is written exactly as if it were real.

**API** (Application Programming Interface) — the set of web addresses the server offers for data, for example `/api/employees`. The browser asks those addresses for data instead of asking for a whole page.

**URL / route** — the address in the browser bar, e.g. `/employees/7/edit`. In this app, the part after the domain decides which screen you see. A *route* is one rule that maps one URL pattern to one screen.

**localStorage** — a small notepad the browser keeps on your own computer, per website. Anything you write there survives closing the tab. This app uses it to remember who is signed in and which theme you chose.

**JSON** — a plain-text way of writing data that looks like JavaScript objects: `{"name": "Mei", "age": 30}`. It is how data travels between browser and server.

## 1.2 TypeScript words

**TypeScript** — JavaScript with *types* added. You write TypeScript; a tool converts ("compiles") it into plain JavaScript that browsers understand. The whole point is that mistakes are caught while you type instead of when a user clicks.

**Type** — a label saying what kind of value something is: `string` (text), `number`, `boolean` (true/false). If you promise a number and hand over text, the compiler stops you.

**interface** — a description of the *shape* of an object: which fields it must have and of what type. It is a contract only; it disappears completely when the code is compiled. Example from this project:

```
interface Address {  
  line1: string;  
  city: string;  
  state: string;  
  pincode: string;  
}
```

Now anything typed `Address` must have exactly those four text fields.

**type alias** (the `type` keyword) — a nickname for a type. It can also list the only allowed values, which is how this project defines roles:

```
type Role = 'GUEST' | 'EMPLOYEE' | 'MANAGER' | 'ADMIN';
```

The `|` means "or". A `Role` can only ever be one of those four exact strings. Typing `'ADMINN'` becomes an error before the app ever runs.

**class** — a blueprint that bundles data and the functions that work on it. Every screen in this project is a class.

**readonly** — "this can be set once and never changed afterwards". A safety label.

**private / protected / public** — who is allowed to touch a field. `private` = only this class. `protected` = this class and its HTML template. `public` (the default) = anyone. This project uses `protected` for anything the template needs and `private` for everything else.

**Omit<A, 'b'>** — a built-in helper meaning "the type A, but without field b". This project writes `type EmployeeDraft = Omit<Employee, 'id'>`, which means "an employee that does not have an id yet" — exactly what you have before saving a new person, because the id is assigned by the server.

**Partial<A>** — "the type A, but every field is optional". Used when you are updating only some fields.

**Record<K, V>** — "an object whose keys are of type K and whose values are of type V". `Record<Role, number>` means an object with one number for each role.

**Generic** (the `<T>` brackets) — a type with a blank to be filled in later. `Page<Employee>` means "a Page of Employees"; `Page<Announcement>` means "a Page of Announcements". One definition, many uses.

**?.** (optional chaining) — "read this, but if the thing on the left is missing, just give me nothing instead of crashing". `employee?.name` is safe even when `employee` is null.

**??** (nullish coalescing) — "use the left value, unless it is missing, in which case use the right one". `name ?? 'Guest'` .

**...** (spread) — "copy everything out of this". `{ ...person, city: 'Pune' }` means "a new object with all of person's fields, except city is now Pune". The original is untouched — this is how the project changes data without editing it in place.

**async / await / Promise** — a `Promise` is a receipt for a value that will exist later (for example, data still travelling from the server). `await` means "pause here until the receipt is honoured", and a function containing `await` must be marked `async` . It lets slow code be written in a straight line.

**Arrow function** — a short way to write a function: `(x) => x * 2` is a function that doubles its input.

## 1.3 Angular words

**Angular** — the framework this project is built with. A framework is a large pre-written toolkit that gives you structure: how to split a page into pieces, how to fetch data, how to move between screens. You write your parts; Angular runs them.

**Component** — the basic building block. One component = one piece of screen. It has three parts: a *template* (its HTML), *styles* (its CSS), and a *class* (its TypeScript logic and data). A page is components inside components inside components.

**Template** — a component's HTML, with extra Angular syntax mixed in so it can show live data.

**Decorator** — a line beginning with `@` written directly above a class, which tells Angular what the class *is*. `@Component` says "this is a screen piece", `@Injectable` says "this is a shared worker", `@Directive` says "this adds behaviour to an element". Think of it as a label on a box.

**selector** — the HTML tag name you use to place a component. If a component declares `selector: 'app-card'` , you drop it into any template by writing `<app-card></app-card>` .

**Standalone component** — a component that lists its own dependencies in an `imports` array, instead of being registered in a big shared list called an `NgModule`. Everything in this project is standalone except the admin area, which deliberately uses the old `NgModule` style to show both approaches.

**Interpolation** — the `{{ }}` syntax. `{{ name }}` in a template means "print the current value of `name` here". It updates automatically when the value changes.

**Property binding** — square brackets: `[value]="total"`. It means "keep this element's *value* property equal to my *total* variable". Data flows from the class into the HTML.

**Event binding** — round brackets: `(click)="save()"`. It means "when this is clicked, run my *save* function". Data flows from the HTML back into the class. A memory trick: **[square] goes in, (round) comes out**.

**@if / @for** — Angular's built-in template blocks. `@if (x) { ... }` shows the contents only when `x` is true. `@for (item of list; track item.id) { ... }` repeats the contents once per item. `track` tells Angular how to recognise each row so it can move existing rows instead of rebuilding the whole list.

**Directive** — extra behaviour attached to an element that already exists, rather than a new element of its own. Two kinds: **attribute directives** change how an element looks or behaves (a tooltip); **structural directives** add or remove elements from the DOM entirely (show this button only for admins). Structural ones are written with a leading `*`.

**Pipe** — a small formatter used inside a template with the `|` symbol. `{{ salary | currency }}` takes the raw number and prints it as money. Pipes only change what is *displayed*; the underlying data is untouched.

**Service** — a plain class that does work but owns no screen: fetching data, keeping the session, writing logs. Components stay small by handing jobs to services.

**Dependency Injection (DI)** — the system that hands a component the services it needs, instead of the component building them itself. You write `private auth = inject(AuthService)`, which means "Angular, give me the `AuthService`". Angular creates one and keeps reusing the same one.

**In plain terms:** you do not go to the kitchen and cook; you ask for the dish and it arrives. And because everyone asks the same kitchen, everyone gets the same food — which is exactly why signing in on one screen is instantly known by every other screen.

**Provider** — the recipe that tells DI how to make something. `useClass` = build this class. `useValue` = just use this fixed value. `useFactory` = run this function to build it. `useExisting` = do not build anything, point at a thing that already exists.

**Injectable / providedIn: 'root'** — the label on a service saying "DI may create me, and there should be exactly one of me for the whole application". One shared copy is called a *singleton*.

**InjectionToken** — a unique name-tag used to inject something that is not a class (a settings object, a list of flags). You cannot write `inject(SomeInterface)` because interfaces vanish at compile time, so you make a token instead and inject that.

**Router** — the part of Angular that watches the URL and swaps the right screen into the page, without ever reloading the browser. That is what makes it a *single-page application*: one page load, many screens.

**router-outlet** — the placeholder tag in a template that says "whichever screen the URL selects, render it right here".

**Lazy loading** — not downloading a screen's code until someone actually visits it. Written as `loadComponent: () => import('...')`. It makes the first page load much faster, because the admin area (which most people never open) is never sent to most people.

**Guard** — a function that runs *before* a route is allowed, and answers yes or no. Returning `true` lets the navigation through; returning a redirect sends the user somewhere else. This is how "you must sign in first" is enforced.

**Resolver** — a function that fetches data *before* a screen is shown, so the screen never has to render an empty state while waiting.

**Lifecycle hook** — a method Angular calls at a fixed moment in a component's life. `ngOnDestroy()` runs just before the component is removed — the place to clean up timers. `ngAfterViewInit()` runs once the component's HTML actually exists.

**Change detection** — Angular's process of checking "has any displayed value changed, and does the screen need updating?". `OnPush` is a setting meaning "only re-check me when something I actually depend on changes" — faster than checking everything constantly.

**Zoneless** — this app runs without Zone.js, the older library that detected changes by secretly wrapping every browser function. Instead, signals report their own changes. Less magic, less work, faster app.

**SSR** (Server-Side Rendering) — building the first version of the page on the server and sending finished HTML, so the user sees content immediately instead of a blank screen.

**Hydration** — what happens next: the Angular app starts up in the browser and "adopts" the HTML the server already sent, attaching the click handlers to it rather than throwing it away and rebuilding.

**ViewEncapsulation** — the rule about whether a component's CSS can escape. *Emulated* (the normal default) = my styles stay inside me. *None* = my styles leak into the whole page. *ShadowDom* = a sealed bubble; nothing gets in or out.

**Content projection** — letting a parent pass HTML *into* a child component, which the child places using `<ng-content />`. It is how one Card component can wrap any content at all.

**Reactive forms** — building a form in TypeScript ( `FormBuilder` , `FormGroup` , `FormControl` ) rather than in HTML, so the rules and the current values live in code where they can be tested.

**Validator** — a rule attached to a form field, e.g. `Validators.required`. A field that breaks a rule is *invalid*. *touched* means the user has visited and left it — the project only shows errors on touched fields, so you are not scolded before you type.

**Sanitisation** — Angular automatically stripping dangerous parts out of HTML that came from a user (scripts, `onerror=` handlers) before putting it on screen. It prevents an attack called XSS, where someone types code into a text box and it runs on other people's computers.

## 1.4 Signals and observables

These two are the heart of the project, so they get more space.

**Signal** — a box holding one value, which announces it when the value changes. You *read* it by calling it like a function:

```
const count = signal(0); // create a box holding 0
count();                // read it -> 0
count.set(5);           // put 5 in the box
count.update(n => n + 1); // change it based on what is inside -> 6
```

**In plain terms:** a signal is a value that tells you when it changes. Any template that read it is redrawn automatically. This is why you will never see code in this project that says "now go update the screen" — the screen follows the data by itself.

**Watch out:** the brackets matter. `count` is the box; `count()` is what is inside it. Forgetting the brackets in a template prints something like "function count()" instead of a number.

**computed** — a signal calculated from other signals. It recalculates itself only when one of its ingredients changes, and caches the answer otherwise.

```
const total = computed(() => employees().length);
```

You never set a computed value; you only ever read it. It is derived data, and it can never go stale or disagree with its source.



**effect** — a block of code that re-runs whenever any signal it reads changes. Used for side effects — things outside Angular, such as writing to `localStorage` or changing an attribute on the page. Rule of thumb: use `computed` to *calculate*, use `effect` to *act*.

**asReadonly()** — hands out a version of a signal that can be read but not changed. Services use it so that outsiders can watch the value while only the service itself may set it.

**input() / output()** — how components talk to each other. An *input* is data a parent passes down into a child. An *output* is an event a child sends up to its parent. `input.required<Employee>()` means "I must be given an employee; refuse to compile otherwise".

**Observable** — a stream of values arriving over time, rather than one value now. A signal is a box; an observable is a pipe. HTTP responses, timers and typing events are all observables. Nothing happens until someone *subscribes* — an unsubscribed observable is a tap nobody opened.

**subscribe** — "start listening to this stream, and here is what to do with each value". You may pass three named handlers, called an *Observer*:

```
stream.subscribe({
  next:    (value) => { /* a value arrived */ },
  error:   (err)   => { /* it failed */ },
  complete: ()     => { /* it finished */ },
});
```

**unsubscribe / teardown** — stopping the listening. This matters: a timer that is never stopped keeps running after you leave the screen, quietly wasting battery forever. That mistake is called a *memory leak*, and this project takes deliberate care to avoid it.

**RxJS** — the library that provides observables and the operators below.

**Operator** — a small transformation applied to a stream inside `.pipe(...)`. The ones used in this project:

| OPERATOR                          | WHAT IT DOES, IN PLAIN WORDS                                                                                   |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------|
| <code>map</code>                  | Change each value into something else. "Give me just the items array out of this response envelope."           |
| <code>filter</code>               | Let only some values through. "Only even-numbered ticks."                                                      |
| <code>tap</code>                  | Peek at a value without changing it — used for logging.                                                        |
| <code>takeUntil</code>            | Keep going until another stream fires, then stop. The standard way to end a subscription when a screen closes. |
| <code>debounceTime</code>         | Wait for a pause. "Only search once they have stopped typing for 250ms."                                       |
| <code>distinctUntilChanged</code> | Ignore a value identical to the previous one.                                                                  |
| <code>catchError</code>           | Handle a failure in the stream.                                                                                |
| <code>finalize</code>             | Run this at the very end, whether it succeeded or failed — like a "finally" block.                             |
| <code>shareReplay</code>          | One shared stream for many listeners, instead of restarting it per listener.                                   |
| <code>mergeMap</code>             | For each value, start another stream and merge the results in.                                                 |
| <code>delay</code>                | Hold each value back for a while — used here to fake network slowness.                                         |

**Subject** — an observable you can push values into by hand. Used in this project as an on/off switch: a `destroy$` Subject that fires once when a screen closes, which every `takeUntil` is watching.

**firstValueFrom** — converts an observable into a Promise by taking its first value. It lets the stores use easy `await` syntax over HTTP calls.

## 1.5 HTTP words

**HTTP** — the language browsers and servers speak. A request goes out, a response comes back.

**HTTP methods** — the verb of the request: **GET** = read something, **POST** = create something, **PUT** = update something, **DELETE** = remove something.

**Status code** — the three-digit result. **200** fine, **201** created, **204** done with nothing to return, **403** you are not allowed, **404** not found, **409** conflict, **500** the server broke.

**Header** — extra labelled information attached to a request, such as who you are. This project attaches `Authorization` and `X-Portal-Role` headers.

**Token** — a string that proves you are signed in, sent with every request so the server knows who is asking.

**Interceptor** — a function every HTTP request passes through on its way out and every response passes through on its way back. Interceptors are stacked, so a request travels down through all of them and the response travels back up.

**In plain terms:** think of airport checkpoints. Your request walks through check-in, security and the gate on the way out, and back through them in reverse on the way home. Each checkpoint can stamp it, time it, redirect it, or turn it back.

**Cache** — keeping a copy of an answer so the same question does not have to be asked twice. A *TTL* (time to live) is how long the copy stays trustworthy — 30 seconds here.

## 2. The big picture

### What is this app?

It is an internal website for a company's HR department — the tool staff would open at work to look up colleagues and manage people. It has eight screens:

| SCREEN      | WHAT YOU DO THERE                                                                            |
|-------------|----------------------------------------------------------------------------------------------|
| Login       | Sign in. Three demo accounts let you try each role.                                          |
| Dashboard   | The home page: headcount, attendance today, birthdays, announcements, recent activity.       |
| Employees   | The staff directory. Search, filter, sort, page through, view one person, add, edit, delete. |
| Attendance  | Who was present, late or absent on a chosen day.                                             |
| Leave       | Apply for holiday; managers approve or reject requests.                                      |
| Departments | The list of departments, with headcount, and a detail panel per department.                  |
| Admin       | Five sub-pages: overview, announcements, system health, audit log, settings.                 |
| Settings    | Your own profile, theme choice, change password.                                             |

### The four roles

Everything in the app is shaped by *who you are*. There are four roles, ranked:

```
GUEST (0) < EMPLOYEE (1) < MANAGER (2) < ADMIN (3)
```

Each role can do everything the ones below it can, plus more. Giving each role a number is what makes the check simple: "is my number at least as big as the number required?" That single comparison lives in `AuthService.hasRole()` and is used everywhere.

| ROLE     | CAN DO                                                                                                                                         |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------|
| GUEST    | See the dashboard only. Not signed in.                                                                                                         |
| EMPLOYEE | See the directory, and <i>their own</i> attendance and leave. Apply for leave.                                                                 |
| MANAGER  | Everything above, plus: see everyone's attendance and leave, approve/reject leave, add and edit employees, see salaries, reach the admin area. |
| ADMIN    | Everything, plus: delete employees, manage departments, and open admin settings.                                                               |

**The most important idea in the whole project:** the role is checked in three separate places — the route guard (can you open this URL?), the structural directive (does this button even exist in the page?), and the fake API (will the data layer obey this request?). Hiding a button is only politeness; the check on the data side is the real lock. Anyone can open the browser's developer tools and unhide a button, but they cannot talk the API into deleting a record their role is not allowed to delete.

## The folder map

```
src/
  main.ts           the very first line of code that runs
  index.html        the empty page Angular fills in
  styles.css        colours and shared styling for the whole app
  server.ts, main.server.ts the server-side rendering entry points

app/
  app.ts / app.html the frame: header, nav, footer
  app.config.ts     the app's setup list
  app.routes.ts     which URL shows which screen

core/
  models/           the brain. No screens live here at all.
  tokens/           the shapes of the data
  services/         named settings and contracts
  state/            the workers: auth, http, theme, logging
  interceptors/     the memory: stores holding the app's data
  guards/, resolvers/ the HTTP checkpoints
                    route gatekeepers and data pre-loaders

shared/
  components/       reusable UI pieces used by many screens
  directives/       card, dialog, chart, toast, bell, badge
  pipes/            tooltip, role check, click-outside
                    initials, tenure

features/           the actual screens
  dashboard/ employees/ attendance/ leave/
  departments/ admin/ auth/ settings/ misc/

testing/            helpers used only by tests
```

## The one rule about folders

Imports may only travel *downwards*:

```
features/    the screens
|  may use
v
shared/      reusable UI
|  may use
v
core/        data, services, no UI at all
```

So a screen may use a card; a card may use a service; but `core` may never import a button, and `shared` may never import a screen. Why does this matter? Because it means you can read `core` without knowing anything about the screens, and you can delete a screen without breaking anything below it. When two layers may call each other freely, you eventually cannot change either one safely — every file becomes tangled with every other file.

### 3. One click, start to finish

---

This is the section to understand properly. We will follow one action — an admin deleting an employee — through every layer of the app. Every file mentioned here is explained in full later; right now, just watch the journey.

#### Step 1 — The button exists at all

```
<button *appHasRole="'ADMIN'" (click)="askToDelete(employee)">Delete</button>
```

`*appHasRole="'ADMIN'"` is a structural directive. If you are not an admin, this button is not hidden with CSS — it is *never created*. It does not exist in the DOM. A manager inspecting the page with developer tools finds nothing to unhide.

#### Step 2 — The click is caught

`(click)="askToDelete(employee)"` runs a method on the `EmployeeList` class. That method does not delete anything. It stores the employee in a signal:

```
this.pendingDelete.set(employee);
```

Because `pendingDelete` is a signal, and the template has `@if (pendingDelete())` around a confirmation dialog, the dialog appears on screen by itself. Nobody wrote "now show the dialog" — the screen simply follows the data.

#### Step 3 — The user confirms

The dialog's Confirm button calls `confirmDelete()`, which clears the signal (dialog disappears) and calls the store:

```
await this.store.remove(employee.id);
```

#### Step 4 — The store takes over

`EmployeeStore` is the app's memory for employees. Its `remove` method first records that something is in progress, then asks the service to do the work:

```
patchState(store, { loading: true, error: null });  
await firstValueFrom(service.remove(id));
```

#### Step 5 — The service builds the request

`EmployeeService.remove()` turns that into an actual HTTP request:

```
this.http.delete<void>(`/api/employees/${id}`)
```

#### Step 6 — Down through the five checkpoints

The request now travels through the interceptor chain, outermost first:

```
DELETE /api/employees/7
|
1. profiling  starts a stopwatch
|
2. auth       attaches Authorization + X-Portal-Role: ADMIN
|
3. cache      not a GET, so it wipes the cached lists
|
4. error      stands ready to catch a failure
|
5. mock backend answers from memory instead of a real server
```

Each one is a small function that can inspect the request, change it, or answer it outright. The order is chosen deliberately: profiling wraps everything so it can time even a cache hit; auth runs before the cache so cached entries carry the right role; the mock backend is last so everything else behaves exactly as it would against a real server.

## Step 7 — The fake backend decides

The mock backend reads the role header and enforces the real rule:

```
if (role !== 'ADMIN') {
  return forbidden(req.url, 'Only administrators may delete employee records.');
```

```
}
db.employees.splice(index, 1);
return noContent(); // 204
```

**This is the real lock.** Even if someone forced the hidden button to appear, the request would come back 403 Forbidden. That is the difference between hiding a button and enforcing a rule.

## Step 8 — Back up through the checkpoints

The 204 response travels back up the same chain in reverse. The error interceptor sees no error and passes it through. The profiling interceptor stops its stopwatch and records the timing — which is what later appears on the admin System Health page.

## Step 9 — The store updates its memory

```
patchState(store, (state) => ({
  employees: state.employees.filter((e) => e.id !== id),
  loading: false,
}));
notifications.success('Employee removed.');
```

Note `filter`: it builds a *new* array without that person rather than editing the old one. A new array is a new value, so the signal genuinely changed, so everything watching it is told.

## Step 10 — Everything redraws itself

Because one signal changed, all of this happens with no further code:

- the table row disappears;
- the "Total employees" tile counts down — it reads a `computed` built on that same list;

- the dashboard's department bar chart reshapes;
- a green toast slides in at the corner;
- the audit log on the admin page gains a line.

**The whole point:** nine of those ten steps are about data. Only the last one is about the screen, and it is automatic. This is what people mean by "reactive": you change the data, and the interface is simply a picture of the data, so the picture updates itself.

**Worth noticing:** the same journey happens for every action in the app — approving leave, publishing an announcement, creating a department. Component → store → service → interceptors → backend, and back. Once you have seen it once, every other feature is the same shape with different nouns.



## 4. The starting files

These few files run before anything else. They set the app up and then get out of the way.

### src/index.html

The only real HTML page in the entire project.

**Its job:** be an empty shell that Angular fills in.

```
<body>
  <app-root></app-root>
</body>
```

`<app-root>` is not a real HTML tag — it is the selector of our top component. Angular finds this tag and builds the entire application inside it. Everything you ever see on screen lives inside those two tags.

| LINE                                          | WHAT IT MEANS                                                                                                                               |
|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <code>&lt;base href="/"&gt;</code>            | Tells the browser that all addresses start from the site root. Without it, refreshing on a deep link like <code>/employees/7</code> breaks. |
| <code>&lt;meta name="viewport" ...&gt;</code> | Makes the page size itself correctly on a phone instead of rendering a shrunken desktop page.                                               |

### src/main.ts

Seven lines. The first application code that runs in the browser.

```
bootstrapApplication(App, appConfig).catch((err) => console.error(err));
```

| LINE                                   | WHAT IT MEANS                                                                          |
|----------------------------------------|----------------------------------------------------------------------------------------|
| <code>bootstrapApplication(...)</code> | "Start Angular." <i>Bootstrap</i> just means start up.                                 |
| <code>App</code>                       | Which component is the root — the one that goes inside <code>&lt;app-root&gt;</code> . |
| <code>appConfig</code>                 | The settings list (next file): routing, HTTP, interceptors, providers.                 |
| <code>.catch(...)</code>               | If startup itself fails, print the reason rather than failing silently.                |

### src/app/app.config.ts

The application's setup list. Arguably the most important configuration file here.

**Its job:** switch on every Angular feature the app uses, and register every shared thing DI must know how to build.

| LINE                                                                                       | WHAT IT MEANS                                                                                                                                                                                                                                                                                |
|--------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>provideZonelessChangeDetection()</code>                                              | Run without Zone.js. Signals report their own changes, so Angular does not have to watch every browser function. Faster, and far less magic.                                                                                                                                                 |
| <code>provideRouter(routes, ...)</code>                                                    | Turn on URL-based navigation, using the route table from <code>app.routes.ts</code> .                                                                                                                                                                                                        |
| <code>withInMemoryScrolling({...})</code>                                                  | Restore scroll position when you press Back, and jump to <code>#anchors</code> . Small touch, noticeably better feel.                                                                                                                                                                        |
| <code>withComponentInputBinding()</code>                                                   | Lets a URL parameter arrive straight into a component's <code>input()</code> , with no manual reading of the route.                                                                                                                                                                          |
| <code>PathLocationStrategy</code>                                                          | Clean URLs ( <code>/employees</code> ) rather than hash URLs ( <code>/#/employees</code> ).                                                                                                                                                                                                  |
| <code>provideHttpClient(withFetch(), ...)</code>                                           | Turn on the HTTP service, using the modern <code>fetch</code> API underneath (which SSR needs).                                                                                                                                                                                              |
| <code>withInterceptors([...])</code>                                                       | The five checkpoints, <b>in order, outermost first</b> . Order is a deliberate design decision, not an accident — see section 5.5.                                                                                                                                                           |
| <code>provideClientHydration(withEventReplay())</code>                                     | Adopt the server-rendered HTML instead of rebuilding it. <i>Event replay</i> means a click made before the app finished loading is remembered and replayed afterwards, so early clicks are not lost.                                                                                         |
| <code>ConsoleLogger</code><br><code>{ provide: Logger, useExisting: ConsoleLogger }</code> | Two names, <b>one object</b> . <code>useExisting</code> points the abstract <code>Logger</code> name at the concrete logger that already exists. Using <code>useClass</code> here would quietly create a <i>second</i> logger, and half the app's log lines would vanish into the wrong one. |
| <code>...provideFeatureFlags(...)</code>                                                   | Register on/off switches for unfinished features. The <code>...</code> spreads an array of providers into this list.                                                                                                                                                                         |

## src/app/app.routes.ts

The map from URL to screen.

```
{
  path: 'employees',
  canActivate: [authGuard],
  loadChildren: () => import('@features/employees/employees.routes')
    .then((m) => m.EMPLOYEE_ROUTES),
}
```

| LINE                                            | WHAT IT MEANS                                                                                                                             |
|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <code>path: 'employees'</code>                  | Matches the URL <code>/employees</code> .                                                                                                 |
| <code>canActivate: [AuthGuard]</code>           | Before opening, ask the guard. If it says no, the navigation is cancelled or redirected.                                                  |
| <code>loadChildren: () =&gt; import(...)</code> | Lazy loading. The arrow function means the import does not run now — only when someone actually visits. That code is a separate download. |
| <code>redirectTo: 'dashboard'</code>            | Visiting <code>/</code> sends you to the dashboard.                                                                                       |
| <code>path: '**'</code>                         | The wildcard: any URL nothing else matched. Shows the 404 page. It must be last, because the router takes the <i>first</i> match.         |
| <code>title: '...'</code>                       | Sets the browser tab title per screen.                                                                                                    |

**Careful:** route order matters. If `'**'` were written first, it would swallow every URL and no screen would ever load.

## src/app/app.ts & app.html

The root component — the frame around every screen.

**Its job:** draw the top bar (logo, navigation, theme button, notification bell, account menu) and leave a hole where the current screen goes.

| LINE                                                           | WHAT IT MEANS                                                                                                                                          |
|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>changeDetection: OnPush</code>                           | Only re-check this component when something it depends on actually changes.                                                                            |
| <code>imports: [RouterOutlet, RouterLink, ...]</code>          | Standalone components must list every directive, pipe and component their template uses. Forget one and the template silently does nothing.            |
| <code>protected readonly auth = inject(AuthService)</code>     | Ask DI for the session service. <code>protected</code> so the HTML may read it.                                                                        |
| <code>menuOpen = signal(false)</code>                          | Is the mobile menu open? A signal, so the template follows it automatically.                                                                           |
| <code>visibleNav = computed(() =&gt; NAV.filter(...))</code>   | Only the nav links this role may use. Because it is <code>computed</code> over <code>auth.hasRole</code> , switching role instantly reshapes the menu. |
| <code>constructor() { void this.notifications.load(); }</code> | Fetch notifications once at start-up. <code>void</code> says "I know this returns a Promise and I am deliberately not waiting for it".                 |
| <code>&lt;router-outlet /&gt;</code>                           | The hole. Whatever the URL selects is drawn here.                                                                                                      |
| <code>&lt;a class="skip-link" href="#main"&gt;</code>          | An accessibility feature: a keyboard user can jump straight past the navigation to the content.                                                        |
| <code>i18n="@app.brand"</code>                                 | Marks text for translation. This app ships in English and French.                                                                                      |

## src/server.ts, main.server.ts, app.config.server.ts, app.routes.server.ts

The four server-side rendering files.

**Their job:** build the first version of a page on the server so the user sees content immediately.

| FILE                              | WHAT IT DOES                                                                                                                   |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <code>server.ts</code>            | A small Express web server. It serves the built files, and hands any other address to Angular to render.                       |
| <code>main.server.ts</code>       | The server twin of <code>main.ts</code> — starts Angular in server mode.                                                       |
| <code>app.config.server.ts</code> | Takes the normal config and merges the server-only extras in. <code>mergeApplicationConfig</code> means nothing is duplicated. |
| <code>app.routes.server.ts</code> | Chooses <i>how</i> each route is rendered.                                                                                     |

Three rendering modes appear in `app.routes.server.ts`, and each is chosen for a reason:

| MODE      | MEANING                                                         | USED FOR                                                                                                         |
|-----------|-----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Prerender | Built once at build time, like a static file. Fastest possible. | The dashboard and login — same for everyone.                                                                     |
| Server    | Built fresh on every request.                                   | Employees, attendance, leave — the content depends on data.                                                      |
| Client    | Not rendered on the server at all; built in the browser.        | Admin and settings — private, and the badge component needs a real Shadow DOM, which does not exist on a server. |

## 5. core/ — the brain of the app

Nothing in `core/` draws anything. It holds data shapes, workers and memory. If you deleted every screen, this folder would still make sense on its own.

### 5.1 core/models — the shapes of the data

These files contain *no runnable logic* for the most part — they describe what an employee *is*. Their value is that the compiler can then check every other file against them.

#### core/models/employee.model.ts

**Its job:** define what an employee is, plus a few small helper functions about employees.

| LINE                                                                              | WHAT IT MEANS                                                                                                                                                                                           |
|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>type Role = 'GUEST'   'EMPLOYEE'   'MANAGER'   'ADMIN'</code>               | The only four roles that can exist. A typo becomes a compile error.                                                                                                                                     |
| <code>const ROLES: readonly Role[] = [...] as const</code>                        | The same four as a real array you can loop over in a dropdown. The <code>type</code> is for the compiler; the <code>const</code> is for the running program. Both are needed.                           |
| <code>type EmployeeStatus = 'ACTIVE'   'ON_LEAVE'   'PROBATION'   'EXITED'</code> | The four states a person can be in.                                                                                                                                                                     |
| <code>interface Employee { ... }</code>                                           | The full record: personal fields, professional fields, address, bio, avatar colour. Grouped with comment headers so it reads like a form.                                                               |
| <code>readonly id: number</code>                                                  | The id is assigned once and never changed.                                                                                                                                                              |
| <code>managerId: number   null</code>                                             | Points at another employee — or <code>null</code> for the person at the top who reports to nobody.                                                                                                      |
| <code>type EmployeeDraft = Omit&lt;Employee, 'id'&gt;</code>                      | An employee-in-progress, before saving. The server assigns the id, so the draft must not have one. This one line prevents an entire class of bug.                                                       |
| <code>interface EmployeeFilter / EMPTY_FILTER</code>                              | The search and filter settings, plus the "nothing selected" starting value used when clearing filters.                                                                                                  |
| <code>function fullName(e)</code>                                                 | Joins first and last name. Written once so 30 files do not each invent their own version.                                                                                                               |
| <code>function nextEmployeeCode(existing)</code>                                  | Finds the highest existing code number and adds one, producing <code>EMP017</code> . <code>replace(/\D/g, '')</code> strips every non-digit; <code>padStart(3, '0')</code> pads to three digits.        |
| <code>function birthdayWithinDays(dob, days)</code>                               | Is this birthday coming up soon? It rebuilds the date using <i>this</i> year, and if that has already passed, adds a year — because a birthday repeats and the year in the record is the year of birth. |

## core/models/hr.model.ts

**Its job:** the same treatment for attendance, leave, departments, documents and notifications.

| LINE                                                                                                          | WHAT IT MEANS                                                                                                                                                                                      |
|---------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>interface AttendanceRecord</code>                                                                       | One person, one day: check-in time, check-out time, status. Times are <code>string   null</code> because an absent person has no check-in.                                                         |
| <code>interface LeaveRequest</code>                                                                           | A holiday request: type, dates, day count, reason, status, who decided it.                                                                                                                         |
| <code>type LeaveDraft = Omit&lt;LeaveRequest, 'id'   'status'   'appliedOn'   'decidedBy'   'days'&gt;</code> | What <i>you</i> fill in. Everything omitted is decided by the system, not by you — you cannot submit a request that is already approved.                                                           |
| <code>interface LeaveBalance</code>                                                                           | How many days of each type you have left.                                                                                                                                                          |
| <code>function countLeaveDays(from, to)</code>                                                                | Counts working days between two dates. It walks day by day and skips <code>day === 0</code> (Sunday) and <code>day === 6</code> (Saturday), so a Friday-to-Monday request counts as 2 days, not 4. |

## core/models/announcement.model.ts

**Its job:** describe a company notice.

Note `bodyHtml: string` — the body is rich text written by a manager, so it may contain HTML. That makes it *untrusted input*, which is why it is sanitised before display (see section 7.6). `pinned: boolean` keeps important notices at the top.

## core/models/api.model.ts

**Its job:** describe the envelopes that wrap data going to and from the API.

| LINE                                                               | WHAT IT MEANS                                                                                                                                                  |
|--------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>interface Page&lt;T&gt; { items: T[]; total: number }</code> | Every list endpoint returns this shape. Generic, so <code>Page&lt;Employee&gt;</code> and <code>Page&lt;LeaveRequest&gt;</code> both work from one definition. |
| <code>interface ApiError</code>                                    | One tidy error shape. The error interceptor converts every kind of failure into this, so screens handle one shape instead of five.                             |
| <code>interface HttpTiming</code>                                  | One row of the performance table: method, URL, how long it took, status.                                                                                       |
| <code>status: number   'CACHE'   'ERROR'</code>                    | Either a real HTTP number, or one of two special words this app adds.                                                                                          |

## 5.2 core/tokens — named settings and contracts

A reminder from the dictionary: you can inject a class by name, but you cannot inject an interface, because interfaces vanish when the code compiles. So for anything that is not a class, you create an `InjectionToken` — a unique name-tag DI can recognise.

## core/tokens/app-config.token.ts

**Its job:** hold the app's settings in one injectable place.

```
export const APP_CONFIG = new InjectionToken<AppConfig>('APP_CONFIG', {
  providedIn: 'root',
  factory: () => ({
    appName: 'Employee Management Portal',
    apiBaseUrl: '/api',
    httpCacheTtlMs: 30_000,
    defaultPageSize: 10,
  }),
});
```

| LINE                                                           | WHAT IT MEANS                                                                                                                                                |
|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>new InjectionToken&lt;AppConfig&gt;('APP_CONFIG')</code> | Create the name-tag. The text is only for error messages.                                                                                                    |
| <code>providedIn: 'root' + factory</code>                      | "If nobody supplies a value, build the default by running this function." This means the settings exist automatically — and can still be replaced in a test. |
| <code>30_000</code>                                            | 30000. The underscore is just a readability separator, ignored by the compiler.                                                                              |

Any file can then write `private config = inject(APP_CONFIG)` and read `config.defaultPageSize`. Change the page size here and every screen changes at once.

## core/tokens/logger.token.ts

**Its job:** define what a logger must be able to do, without saying how.

This one is unusual: it uses an `abstract class` as the token. An abstract class is a class you cannot create directly — only inherit from. That gets you two things at once: a real runtime object DI can use as a token, *and* a contract listing the required methods.

| LINE                                              | WHAT IT MEANS                                                                                                             |
|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>abstract log(level, message): void</code>   | Any real logger must implement this. No default given.                                                                    |
| <code>debug() / info() / warn() / error()</code>  | Convenience methods with real bodies, each just calling <code>log</code> . Written once here so no subclass repeats them. |
| <code>LOG_SINK token, factory: () =&gt; []</code> | A single shared array every logger writes into — which is exactly what the admin Audit Log page reads and displays.       |

## core/tokens/feature-flags.token.ts

**Its job:** allow unfinished features to be switched on and off.

```
return flags.map((flag) => ({ provide: FEATURE_FLAGS, useValue: flag, multi: true }));
```

`multi: true` is the interesting part. Normally a second provider for the same token *replaces* the first. With `multi`, each one *appends to a list* instead. So the root app contributes two flags, the admin module contributes two more, and anyone injecting `FEATURE_FLAGS` receives all four merged together. A lazy-loaded feature can register its own flags without ever touching the root configuration file.

### `core/tokens/analytics.token.ts`

**Its job:** demonstrate an optional dependency.

This token is deliberately *never provided*. Anything that wants it must ask with `inject(ANALYTICS, { optional: true })`, which returns `null` instead of crashing. It shows how a system copes with a service that may or may not be configured in a given deployment.



## 5.3 core/services — the workers

A service is a class that does a job but owns no screen. Every one of these is marked `@Injectable({ providedIn: 'root' })`, meaning there is exactly one of it for the whole app.

### core/services/auth.service.ts

Who is signed in. Probably the most-used file in the project.

**Its job:** remember the current session, expose the role as signals, and answer the question "is this person allowed?"

| LINE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | WHAT IT MEANS                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>const RANK: Record&lt;Role, number&gt; = {<br/>  GUEST: 0, EMPLOYEE: 1, MANAGER: 2, ADMIN:<br/>  3 }<br/><br/>private session = signal&lt;Session   null&gt;<br/>(this.restore())<br/><br/>readonly currentSession =<br/>  this.session.asReadOnly()<br/><br/>isAuthenticated = computed(() =&gt;<br/>  this.session() !== null)<br/><br/>role = computed(() =&gt;<br/>  this.session()?.role ?? 'GUEST')<br/><br/>login(displayName, role, remember)<br/><br/>hasRole(required)<br/><br/>private isBrowser =<br/>  isPlatformBrowser(inject(PLATFORM_ID))<br/><br/>try { ... } catch { ... } around storage<br/><br/>restore() returns null on the server</pre> | <p>Turns roles into numbers so they can be compared with <code>&gt;=</code>. This is the trick that makes the whole permission system one line long.</p> <p>The one piece of truth: the current session, or <code>null</code> when signed out. It starts by trying to restore a remembered session from <code>localStorage</code>.</p> <p>Everyone else may <i>read</i> the session; only this service may change it. A locked display case.</p> <p>Derived, never stored. It cannot disagree with the session, because it <i>is</i> the session, re-expressed.</p> <p>Your role, defaulting to GUEST when signed out. Note <code>?.</code> and <code>??</code> working together: safe read, then a fallback.</p> <p>Builds a session, stores it in the signal, and writes it to <code>localStorage</code> <i>only</i> if "remember me" was ticked.</p> <p>The permission check: <code>RANK[this.role()] &gt;= RANK[required]</code>. Because it reads the <code>role</code> signal, every guard, directive and template using it re-evaluates the instant the role changes.</p> <p>Am I running in a browser or on the server? Vital, because <b>localStorage does not exist on a server</b> — touching it there would crash the render.</p> <p>Private browsing mode can block storage entirely. The app treats that as "just do not remember me" rather than crashing.</p> <p>Deliberate. Server-rendered HTML must never assume someone is signed in, or one user's page could be cached and shown to another.</p> |

### core/services/employee.service.ts

**Its job:** the HTTP calls about employees — and a teaching example of building an observable by hand.

| LINE                                                        | WHAT IT MEANS                                                                                                                                                                                                                              |
|-------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>list(filters)</code>                                  | Builds query parameters, then <code>.pipe(map((page) =&gt; page.items))</code> unwraps the envelope so callers get a plain array rather than <code>{ items, total }</code> .                                                               |
| <code>params = params.set('search', ...)</code>             | Notice the reassignment: <code>HttpParams</code> is immutable, so <code>.set()</code> returns a <i>new</i> object rather than editing the old one. Forgetting the <code>params =</code> is a classic bug — the filter silently disappears. |
| <code>create / update / remove</code>                       | POST, PUT and DELETE. Each uses <code>tap(...)</code> to write a log line without altering the data flowing through.                                                                                                                       |
| <code>headcountFeed()</code>                                | An observable written from scratch rather than with a helper. The function you pass is the producer: it starts a <code>setInterval</code> and calls <code>subscriber.next(...)</code> on each tick.                                        |
| <code>return () =&gt; { clearInterval(handle); }</code>     | The <b>teardown</b> . Whatever you return from that function is run automatically on unsubscribe, completion or error. This is what stops the timer — without it, the interval would run forever after you left the page.                  |
| <code>shareReplay({ bufferSize: 1, refCount: true })</code> | Share one stream between all listeners rather than starting a new timer each time. <code>refCount: true</code> means "when the last listener leaves, actually stop".                                                                       |
| <code>evenHeadcountLabels(notifier)</code>                  | Composition: <code>filter</code> keeps even ticks, <code>map</code> formats them as text, <code>takeUntil(notifier)</code> ends the whole thing when the notifier fires.                                                                   |

## core/services/headcount-feed.service.ts

**Its job:** the live "online now" number on the dashboard.

Same hand-written-observable pattern, used for real this time. It emits immediately, then every five seconds. `distinctUntilChanged` suppresses repeats, so if the number has not moved, nothing is pushed and nothing redraws. The teardown clears the timer the moment you navigate away from the dashboard.

## core/services/hr.service.ts

**Its job:** one thin HTTP client for attendance, leave, departments, documents and notifications.

```
const items = <T>() => map((page: Page<T>) => page.items);
```

A tiny helper defined once at the top, then reused in every method: it unwraps the `{ items, total }` envelope. Without it, that same `map` line would be repeated about ten times in the file.

| METHOD                                                                                 | WHAT IT DOES                                                                                            |
|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <code>attendance(filters)</code>                                                       | GET attendance rows, filtered by date, person or status.                                                |
| <code>applyForLeave(draft)</code>                                                      | POST a new leave request.                                                                               |
| <code>decideLeave(id, approve, by)</code>                                              | POSTs to <code>/leave/7/approve</code> or <code>/leave/7/reject</code> — the action is part of the URL. |
| <code>departments(), createDepartment(), updateDepartment(), deleteDepartment()</code> | Full create-read-update-delete for departments.                                                         |
| <code>markAllNotificationsRead()</code>                                                | One request that clears every unread notification.                                                      |

## core/services/announcement.service.ts

**Its job:** list, create and delete announcements. The simplest service in the project — a good one to read first.

## core/services/theme.service.ts

**Its job:** remember whether you want light, dark, or whatever your computer uses.

| LINE                                                                        | WHAT IT MEANS                                                                                                                                                                                                             |
|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>type ThemePreference = 'system'   'light'   'dark'</code>             | Three choices, not two. "System" means "follow my operating system", which is not the same as light.                                                                                                                      |
| <code>constructor() { effect(() =&gt; this.apply(this.current())); }</code> | An <code>effect</code> : whenever the preference signal changes, re-run <code>apply</code> . This is the correct use of an effect — touching the world outside Angular (the page element and <code>localStorage</code> ). |
| <code>root.removeAttribute('data-theme')</code>                             | For "system", stamp <i>nothing</i> on the page, so the CSS <code>prefers-color-scheme</code> rule takes over naturally.                                                                                                   |
| <code>root.setAttribute('data-theme', preference)</code>                    | For an explicit choice, stamp it on. The stylesheet scores that above the media query, so your choice beats the operating system in both directions.                                                                      |
| <code>cycle()</code>                                                        | <code>system → light → dark → system</code> . The modulo <code>% order.length</code> wraps around at the end, so one button covers three states.                                                                          |

## core/services/notification.service.ts

**Its job:** the small coloured messages ("toasts") that appear at the corner.

| LINE                                                           | WHAT IT MEANS                                                                                                                                                                    |
|----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>success() / error() / info()</code>                      | Three named ways to raise a message, each calling the private <code>push</code> .                                                                                                |
| <code>this.items.update((list) =&gt; [...list, toast])</code>  | Build a <i>new</i> array with the toast appended. Never <code>list.push(toast)</code> — editing the array in place would not change the signal's value, so nothing would redraw. |
| <code>setTimeout(() =&gt; this.dismiss(toast.id), 4000)</code> | Auto-remove after four seconds, so messages do not pile up forever.                                                                                                              |

## core/services/logger.service.ts

**Its job:** record what happened, so the audit page has something to show.

| LINE                                                                 | WHAT IT MEANS                                                                                                                                                           |
|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>class ConsoleLogger extends Logger</code>                      | The real implementation of the abstract contract. Its <code>scope</code> is <code>'root'</code> .                                                                       |
| <code>this.sink.unshift(...)</code>                                  | Add to the <i>front</i> of the shared array, so newest appears first.                                                                                                   |
| <code>if (this.sink.length &gt; 50)<br/>this.sink.length = 50</code> | Keep only the last 50 entries. Setting <code>.length</code> truncates an array — a compact way to cap memory use.                                                       |
| <code>class ScopedLogger extends ConsoleLogger</code>                | The same logger but tagged with a different name, e.g. <code>'admin'</code> . The admin area provides this one, so admin actions are labelled as such in the audit log. |
| <code>scopedLoggerFactory(scope)</code>                              | Returns a <i>function that builds</i> a logger — exactly the shape a <code>useFactory</code> provider needs.                                                            |

## core/services/profiling.service.ts

**Its job:** collect the request timings the profiling interceptor measures.

Everything derived is a `computed`: `requestCount`, `averageMs` (which deliberately excludes cache hits, since a cache hit is not a real network measurement), and `cacheHits`. It keeps only the newest 25 rows via `.slice(0, 25)`.

## core/services/employee-seed.ts

**Its job:** the starting data — 16 employees, 7 departments, attendance rows, leave requests, notifications and announcements.

Written as compact tuples (small fixed-length arrays) plus one function that expands each tuple into a full object, rather than 16 hand-typed records. The benefit: when the `Employee` model gains a field, you edit one mapper instead of sixteen objects.

5.4 core/state — the app's memory

A *store* holds data that more than one screen needs, in one place. Without stores, the dashboard and the employee list would each fetch employees separately, and could disagree with each other.

**SignalStore** — a store built from signals, provided by the `@ngrx/signals` library. It is assembled from four building blocks:

| BLOCK                            | MEANING                                                            |
|----------------------------------|--------------------------------------------------------------------|
| <code>withState({...})</code>    | The data it holds, and its starting values.                        |
| <code>withComputed({...})</code> | Values calculated from that data — counts, filtered lists, totals. |
| <code>withMethods({...})</code>  | The only ways the data may be changed.                             |
| <code>withHooks({...})</code>    | Code that runs when the store first starts up.                     |

`patchState(store, { ... })` is how a method changes the data — it updates only the fields you name and leaves the rest alone.

core/state/employee.store.ts

The largest and most important store. Everything about employees flows through it.

| LINE                                                                                                 | WHAT IT MEANS                                                                                                                                                                                                                               |
|------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>signalStore({ providedIn: 'root' }, ...)</code>                                                | One store for the entire app. The dashboard and the directory read the very same list.                                                                                                                                                      |
| <code>interface EmployeeState { employees, filter, selectedId, loading, error, loadedAt }</code>     | Everything the store remembers. Note that <code>loading</code> and <code>error</code> are stored data too — that is what lets any screen show a spinner or a message without inventing its own.                                             |
| <code>filtered = computed(() =&gt; employees().filter((e) =&gt; matches(e, filter())))</code>        | The visible list. It is never stored, only derived — so it can never drift out of step with the filter.                                                                                                                                     |
| <code>function matches(employee, filter)</code>                                                      | The search rule. It builds a "haystack" string of name, code, email and title, lowercases both sides, and checks all filters with <code>&amp;&amp;</code> . Searching for "des" therefore matches a Designer <i>and</i> anyone named Desai. |
| <code>headcountByDepartment</code>                                                                   | Counts people per department using a <code>Map</code> , converts it to an array, sorts biggest first. This one computed feeds both the dashboard chart and the admin overview.                                                              |
| <code>payrollTotal = computed(() =&gt; employees().reduce((sum, e) =&gt; sum + e.salary, 0))</code>  | <code>reduce</code> folds a list into a single value — here, adding up salaries starting from 0.                                                                                                                                            |
| <code>recentJoiners</code>                                                                           | Anyone who joined in the last 90 days. Because dates are stored as <code>YYYY-MM-DD</code> text, they can be compared with <code>&gt;=</code> directly — alphabetical order and chronological order happen to agree in that format.         |
| <code>designations = computed(() =&gt; [...new Set(...)])</code>                                     | A <code>Set</code> keeps only unique values; the spread turns it back into an array. This fills the job-title dropdown with no duplicates.                                                                                                  |
| <code>async load()</code>                                                                            | Sets <code>loading: true</code> , awaits the service, stores the result, sets <code>loading: false</code> . On failure it stores the message in <code>error</code> instead of crashing.                                                     |
| <code>await firstValueFrom(service.list())</code>                                                    | Turns the observable into a Promise so <code>await</code> can be used.                                                                                                                                                                      |
| <code>create / update / remove</code>                                                                | Each calls the API first, then updates the local list only if the API succeeded. That ordering matters: the screen never shows a change the server rejected.                                                                                |
| <code>employees: state.employees.map((e) =&gt; (e.id === id ? updated : e))</code>                   | Replace one item by building a new array. Same rule as before: new value, so the signal fires.                                                                                                                                              |
| <code>byId(id)</code>                                                                                | Look up one person from the list already in memory — no network request needed.                                                                                                                                                             |
| <code>withHooks({ onInit(store) { if (store.employees().length === 0) void store.load(); } })</code> | Load the data automatically the first time anything uses the store, but never twice. This is why no screen has to remember to fetch employees.                                                                                              |

## core/state/leave.store.ts

**Its job:** leave requests and remaining balances.

| LINE                                                                                                       | WHAT IT MEANS                                                                                                                                     |
|------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>await Promise.all([firstValueFrom(service.leave()), firstValueFrom(service.leaveBalances())])</code> | Fetch both at the same time rather than one after the other. Two 180ms calls take 180ms together, not 360ms.                                      |
| <code>decide(id, approve, decidedBy)</code>                                                                | Approve or reject, then re-fetch the balances — because approving leave consumes someone's allowance, and the number on screen must reflect that. |
| <code>pendingCount, approvedCount, rejectedCount</code>                                                    | Simple computed counts driving the tiles at the top of the leave screen.                                                                          |

## core/state/attendance.store.ts

**Its job:** attendance for one selected day.

`defaultDate()` steps backwards from today until it lands on a weekday, so the page opens on a day that actually has data instead of an empty Sunday. `attendanceRate` counts present *and* late people as "here" — late is still present.

## core/state/department.store.ts

**Its job:** the department list, with full create/update/delete.

`byName` is a `computed` returning a `Map` from name to record, so a lookup is instant rather than a search through the array each time.

## core/state/announcement.store.ts

**Its job:** company announcements.

Its `ordered` `computed` does a two-level sort: pinned items first, then newest first within each group. The comparison `a.pinned ? -1 : 1` is how a sort function says "this one comes first".

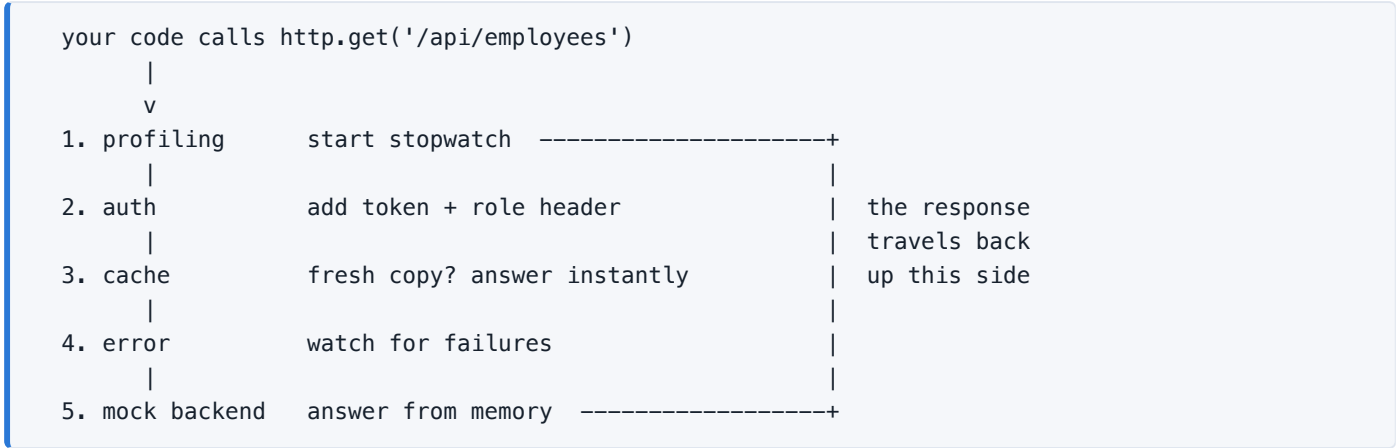
## core/state/notification.store.ts

**Its job:** the bell menu in the header.

| LINE                                                                        | WHAT IT MEANS                                                                                                                                              |
|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>catch { patchState(store, { loading: false }); }</code>               | If loading notifications fails, say nothing. The bell is a nicety — it must never interrupt the user with an error.                                        |
| <code>markRead(id)</code> updates locally <i>first</i> , then calls the API | Called an <i>optimistic update</i> : the badge number drops instantly rather than after a round trip, because the request is virtually certain to succeed. |

### 5.5 core/interceptors — the HTTP checkpoints

Every HTTP request in this app passes through five functions on the way out, and every response comes back through the same five in reverse. Each is small and does exactly one job.



**Why this exact order?** Profiling is first so its stopwatch wraps everything — including a cache hit, which never reaches the network. Auth is before cache so cached entries already carry the right role. Error sits closest to the transport so it sees failures in their raw form. The mock backend is last, standing exactly where a real server would — which means swapping in a real API is deleting one file.

#### core/interceptors/profiling.interceptor.ts

**Its job:** measure how long every request takes.

| LINE                                              | WHAT IT MEANS                                                                                                                                                  |
|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>const startedAt = performance.now()</code>  | A high-precision stopwatch start, accurate to fractions of a millisecond.                                                                                      |
| <code>tap({ next: ..., error: ... })</code>       | Watch what comes back without changing it: record the status code, or note that it failed.                                                                     |
| <code>finalize(() =&gt; { ... })</code>           | Runs at the very end whether it succeeded or failed — so a failed request is still timed and recorded.                                                         |
| <code>durationMs &lt; 1 ? 'CACHE' : status</code> | A clever inference: a response that came back in under a millisecond cannot have crossed a network, so it must have been the cache short-circuiting the chain. |

#### core/interceptors/auth.interceptor.ts

**Its job:** attach proof of identity to every outgoing request.

```
return next(req.clone({
  setHeaders: {
    Authorization: `Bearer ${token}`,
    'X-Portal-Role': auth.role(),
  },
}));
```



| LINE                                        | WHAT IT MEANS                                                                                                                                                            |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>if (!token) return next(req);</code>  | Not signed in? Pass the request along untouched.                                                                                                                         |
| <code>req.clone({ setHeaders: ... })</code> | <b>Requests are immutable</b> — you cannot edit one. You make a copy with the extra headers and send that. Trying to assign a header directly would silently do nothing. |
| <code>next(...)</code>                      | "Hand this on to the next checkpoint." Calling <code>next</code> is what continues the chain; not calling it stops the request dead.                                     |

## core/interceptors/cache.interceptor.ts

**Its job:** avoid asking the same question twice within 30 seconds.

| LINE                                                                                                | WHAT IT MEANS                                                                                                                                                                         |
|-----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>const store = new Map&lt;string, CacheEntry&gt;()</code>                                      | The cache itself, declared at module level (outside the function) so one cache serves the whole app for its whole life.                                                               |
| <code>if (req.method !== 'GET') return next(req).pipe(tap(() =&gt; clearHttpCache()))</code>        | Any write empties the entire cache. Blunt, but always correct — after adding an employee, no stale list can survive.                                                                  |
| <code>const key = req.urlWithParams</code>                                                          | The cache key includes the query string, so <code>?department=Design</code> and <code>?department=Sales</code> are stored separately.                                                 |
| <code>if (hit &amp;&amp; Date.now() - hit.storedAt &lt; ttl) return of(hit.response.clone())</code> | A fresh copy exists, so return it <i>without calling</i> <code>next</code> . The request never reaches the network at all. <code>of(...)</code> wraps a plain value as an observable. |
| <code>CACHE_BYPASS</code> context token                                                             | A per-request flag letting a caller demand a fresh answer, defined in <code>cache.context.ts</code> . Context travels with a request without being sent as a header.                  |

## core/interceptors/error.interceptor.ts

**Its job:** turn every kind of failure into one friendly message, shown once.

| LINE                                               | WHAT IT MEANS                                                                                                                                         |
|----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>function describe(error)</code>              | Translates status codes into human sentences: 0 → "Network unreachable", 403 → "not permitted", 404 → "no longer exists", 500+ → "the server failed". |
| <code>catchError((error) =&gt; { ... })</code>     | Catch anything that failed anywhere below this point in the chain.                                                                                    |
| <code>notifications.error(apiError.message)</code> | Show one toast. Because this is centralised, no individual screen needs its own error-message code.                                                   |
| <code>return throwError(() =&gt; apiError)</code>  | Re-throw after handling, so a screen that wants to react further still can. Handling an error is not the same as swallowing it.                       |

## core/interceptors/mock-backend.interceptor.ts

The biggest file in `core/`. It is a complete fake server.

**Its job:** answer `/api/**` requests from memory, so the app runs with no real backend.

| LINE                                                                             | WHAT IT MEANS                                                                                                                                                  |
|----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>let db: Db = seed()</pre>                                                   | The "database": plain arrays in memory. It resets when you refresh the page.                                                                                   |
| <pre>const clone = (rows) =&gt; rows.map((row) =&gt; ({ ...row })))</pre>        | Hand out <i>copies</i> , never the originals. Otherwise a screen editing its own data would silently rewrite the database.                                     |
| <pre>const path = new URL(req.url, 'http://localhost').pathname</pre>            | Strip the query string so only the path is matched.                                                                                                            |
| <pre>const role = req.headers.get('X-Portal-Role') ?? 'GUEST'</pre>              | Reads the header the auth interceptor added two checkpoints earlier — this is where the two halves of the permission system meet.                              |
| <pre>/^\/api\/employees\/(\d+)\\$/.exec(path)</pre>                              | A regular expression: does the path look like <code>/api/employees/</code> followed by digits? The captured digits become the id.                              |
| <pre>if (role !== 'ADMIN') return forbidden(...)</pre>                           | <b>The real permission check.</b> Hidden buttons are courtesy; this returns 403 regardless of what the interface showed.                                       |
| <pre>const HANDLERS = [...]</pre>                                                | A list of handler functions tried in order. Each returns a response, or <code>null</code> meaning "not mine, try the next". If all decline, the answer is 404. |
| <pre>delay(LATENCY_MS)</pre>                                                     | Adds 180ms of fake network slowness, so loading states and spinners are actually visible during development.                                                   |
| <pre>mergeMap(() =&gt; route(req))</pre>                                         | Defers running the handler until subscription, so the delay applies to the work rather than starting before it.                                                |
| <pre>if (path === '/api/boom') return fail(500, ...)</pre>                       | An endpoint that always fails on purpose, so the admin page can demonstrate error handling.                                                                    |
| <pre>if (inUse) return fail(409, 'This department still has employees...')</pre> | A genuine business rule: you cannot delete a department that still has people in it.                                                                           |

## 5.6 core/guards and core/resolvers

### core/guards/auth.guard.ts

**Its job:** block a route until someone is signed in.

| LINE                                                                                           | WHAT IT MEANS                                                                                                                                                                                                                                                                     |
|------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>if (isPlatformServer(inject(PLATFORM_ID))) return true;</pre>                             | On the server, always allow. The remembered session lives in <code>localStorage</code> , which the server cannot read — refusing there would bounce every signed-in user to the login page. The browser re-runs this guard immediately after loading and redirects properly then. |
| <pre>if (auth.isAuthenticated()) return true;</pre>                                            | Signed in: proceed.                                                                                                                                                                                                                                                               |
| <pre>return router.createUrlTree(['/login'], { queryParams: { redirectTo: state.url } })</pre> | Not signed in: go to login, <i>remembering where you were trying to go</i> . After signing in you land where you meant to, not on a generic home page.                                                                                                                            |

## core/guards/role.guard.ts

**Its job:** block a route unless your role is high enough.

```
export function roleGuard(...allowed: Role[]): CanActivateFn {  
  return () => { /* the actual check */ };  
}
```

This is a **factory**: a function that *returns* a guard. That is why the route can say `roleGuard('MANAGER', 'ADMIN')` — the required role is written in the route table where you can see it, instead of buried inside the guard. One guard file therefore serves every protected route.

## core/guards/unsaved-changes.guard.ts

**Its job:** warn before leaving a half-filled form.

This is a `CanDeactivate` guard — it runs when you try to *leave*, not enter. It asks the component itself: `if (!component.hasUnsavedChanges()) return true;`. Any component implementing the `HasUnsavedChanges` interface can be protected this way, so the guard knows nothing about forms specifically.

## core/resolvers/employee.resolver.ts

**Its job:** fetch one employee *before* their profile page opens.

```
export const employeeResolver: ResolveFn<Employee> = (route) =>  
  inject(EmployeeService).getById(Number(route.paramMap.get('id')));
```

Two lines with a real consequence: the detail screen never renders an empty shell while data loads, so it never needs "if there is no employee yet" handling in its template. The data is guaranteed present before the component is created.

## 6. shared/ — the reusable parts

These are used by many screens. A change here is felt everywhere — which is the point.

### 6.1 shared/components

#### shared/components/card/card.ts

**Its job:** the white panel that almost every block of content sits on.

This is the clearest example of *content projection* in the project. The card provides four slots:

| LINE                                                                                                         | WHAT IT MEANS                                                                                                                                 |
|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>&lt;ng-content select="[card-title]"&gt;&lt;h3&gt;{{ heading() }}&lt;/h3&gt;&lt;/ng-content&gt;</code> | Put whatever the parent marked <code>card-title</code> here — and if the parent gave nothing, fall back to the plain heading text.            |
| <code>&lt;ng-content select="[card-actions]" /&gt;</code>                                                    | A slot for buttons in the top-right corner.                                                                                                   |
| <code>&lt;ng-content /&gt;</code>                                                                            | The catch-all: everything else the parent passed in goes here.                                                                                |
| <code>.card__foot:empty { display: none; }</code>                                                            | The footer collapses itself when nothing was projected into it, so callers never have to declare whether they have a footer.                  |
| <code>encapsulation: ViewEncapsulation.Emulated</code>                                                       | Stated explicitly (though it is the default) because the two components beside it deliberately choose otherwise — the contrast is the lesson. |

**The three encapsulation modes, side by side:**

**Emulated** (card) — my styles cannot escape; page styles can still reach in. The right default.

**None** (print-record) — my styles go into the whole document. Needed there because `@page` print rules cannot be scoped to a component.

**ShadowDom** (badge-widget) — a sealed bubble, isolated both ways. Needed for a component meant to be embedded in a page it does not control.

#### shared/components/stat-tile/stat-tile.ts

**Its job:** one number in a box — "Total employees: 16".

| LINE                                                | WHAT IT MEANS                                                                                                                          |
|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <code>label = input.required&lt;string&gt;()</code> | Must be supplied. Omitting it is a compile error, not a blank tile at runtime.                                                         |
| <code>accent = input('#6366f1')</code>              | Optional, with a default — the coloured stripe down the left edge.                                                                     |
| <code>display = computed(...)</code>                | Formats the value: as currency if <code>currency</code> is true, otherwise with thousands separators via <code>toLocaleString</code> . |
| <code>[style.--tile-accent]=accent()</code>         | Sets a CSS custom property from TypeScript, so the CSS file can use the colour without knowing it in advance.                          |

## shared/components/confirm-dialog/confirm-dialog.ts and dialog-ref.ts

**Its job:** "Are you sure?" before anything destructive.

| LINE                                                                                     | WHAT IT MEANS                                                                                                                                                                                                                 |
|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>viewProviders: [DialogRef]</code>                                                  | Not <code>providers</code> . <code>viewProviders</code> means only things inside <i>this component's own template</i> can inject it — content projected in by a parent cannot. That is precisely the boundary a dialog wants. |
| <code>confirmed = output&lt;void&gt;() / cancelled = output&lt;void&gt;()</code>         | Two events sent up to the parent. The dialog itself never deletes anything; it only reports which button was pressed.                                                                                                         |
| <code>cancelButton = viewChild&lt;ElementRef&gt;('cancelButton')</code>                  | A signal query that finds an element in this component's own template.                                                                                                                                                        |
| <code>ngAfterViewInit() {<br/>  this.cancelButton()?.nativeElement.focus();<br/>}</code> | Move focus into the dialog when it opens — onto <i>Cancel</i> , the least destructive button. A stray Enter keypress should not delete a record.                                                                              |
| <code>@HostListener('document:keydown.escape')</code>                                    | Escape closes the dialog, as every modal is expected to do.                                                                                                                                                                   |
| <code>&lt;button class="backdrop"&gt;</code>                                             | The dark background is a real <code>&lt;button&gt;</code> , not a <code>&lt;div&gt;</code> , so keyboard users can reach and activate it too.                                                                                 |

## shared/components/toast-host/toast-host.ts

**Its job:** draw whatever toasts the NotificationService currently holds.

It is placed once in `app.html`, at the bottom. `role="status"` and `aria-live="polite"` make a screen reader announce new messages without interrupting what the user is currently doing.

## shared/components/notification-bell/notification-bell.ts

**Its job:** the 🛎 in the header, with an unread count and a dropdown.

Reads `NotificationStore`. Uses `(appClickOutside)` to close when you click elsewhere. Each entry is rendered as a link when it has a destination and as a button when it does not — a detail that matters, because a link that goes nowhere is confusing for keyboard and screen-reader users.

## shared/components/bar-chart/bar-chart.ts

**Its job:** the horizontal headcount-by-department chart.

| LINE                                                                     | WHAT IT MEANS                                                                                                                               |
|--------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <code>percent(value)</code>                                              | Each bar's width as a percentage of the largest value, so the biggest bar always fills the space.                                           |
| One colour for all bars                                                  | Deliberate. There is only one measure being shown and each bar is already labelled, so different colours would encode no extra information. |
| <code>&lt;details&gt;&lt;summary&gt;View as table&lt;/summary&gt;</code> | The same numbers as a real table. A chart that can only be read visually excludes people using screen readers.                              |
| <code>[attr.aria-label]="item.label + ': ' + item.value"</code>          | Every bar announces its own value aloud.                                                                                                    |

### shared/components/badge-widget/badge-widget.ts

**Its job:** a small shareable employee badge, designed to be embedded elsewhere.

Uses `ViewEncapsulation.ShadowDom`, giving it a genuine sealed shadow root. A host page cannot restyle it and it cannot leak into a host page — the entire reason such a component exists. This is also why the admin area is client-rendered only: a shadow root has no server-side equivalent.

### shared/components/print-record/print-record.ts

**Its job:** the printable version of an employee record.

Uses `ViewEncapsulation.None`, deliberately. Print rules like `@page { size: A4 }` apply to the whole document — the browser's print box cannot be scoped to one component — so these styles must escape into the document head. Every selector is namespaced under `.print-record` to keep that escape hatch tidy.

## 6.2 shared/directives

A reminder: **attribute** directives change an existing element; **structural** directives (written with `*`) add or remove elements from the page entirely.

### shared/directives/has-role.directive.ts

Structural. The single most important directive in the project.

**Its job:** put an element in the page only if your role is high enough.

```
<button *appHasRole="'ADMIN'; else: denied">Delete</button>
<ng-template #denied>Ask an administrator.</ng-template>
```

| LINE                                                          | WHAT IT MEANS                                                                                                                                                                                                     |
|---------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>private templateRef = inject(TemplateRef)</code>        | The blueprint of the wrapped HTML. The <code>*</code> tells Angular to rewrite the line into an <code>&lt;ng-template&gt;</code> that this directive owns — so the content exists as a plan, not as page content. |
| <code>private viewContainer = inject(ViewContainerRef)</code> | The place where that blueprint can be stamped out into real elements.                                                                                                                                             |
| <code>effect(() =&gt; { ... })</code>                         | Re-runs whenever the role signal changes. Switch role in the admin panel and buttons appear or vanish immediately, with no page reload.                                                                           |
| <code>this.viewContainer.createEmbeddedView(template)</code>  | Actually create the elements.                                                                                                                                                                                     |
| <code>this.viewContainer.clear()</code>                       | Remove them completely.                                                                                                                                                                                           |
| <code>if (this.rendered === next) return;</code>              | Do nothing when the answer has not changed — without this, every unrelated signal change would destroy and rebuild the button, losing focus mid-interaction.                                                      |

**Why not just use CSS to hide it?** Because `display: none` leaves the button in the page. Anyone can open developer tools and un-hide it. With this directive the element genuinely does not exist — and even if someone recreated it by hand, the mock backend still refuses the request.

### `shared/directives/feature-flag.directive.ts`

Structural.

**Its job:** keep unreleased features out of the page until they are switched on.

Same mechanism as `appHasRole`, but it checks the `FEATURE_FLAGS` list instead of the role. Used on the admin System page for an "Export payroll" button that is currently switched off.

### `shared/directives/tooltip.directive.ts`

Attribute.

**Its job:** show a small label on hover or keyboard focus.

| LINE                                                                | WHAT IT MEANS                                                                                                                                                                                 |
|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>@HostListener('mouseenter') and @HostListener('focus')</code> | Two triggers, both shown. Mouse users hover; keyboard users focus. Supporting only hover would exclude keyboard users entirely.                                                               |
| <code>this.renderer.createElement('span')</code>                    | Builds the bubble through <code>Renderer2</code> instead of touching the DOM directly, so the same code runs safely during server-side rendering, where <code>document</code> does not exist. |
| <code>this.renderer.setProperty(bubble, 'textContent', ...)</code>  | <b>textContent, never innerHTML.</b> Text is inserted as text, so a tooltip can never become an injection point.                                                                              |
| <code>@HostBinding('attr.tabindex') tabIndex = '0'</code>           | Makes the element keyboard-reachable, since something that only reveals information on hover is otherwise unreachable.                                                                        |
| <code>@HostBinding('attr.aria-describedby')</code>                  | Links the element to its bubble so a screen reader reads the description too.                                                                                                                 |
| <code>ngOnDestroy() { this.hide(); }</code>                         | Clean-up: never leave a stray bubble behind when the element disappears.                                                                                                                      |

### shared/directives/role-badge.directive.ts

Attribute.

**Its job:** append a coloured role label ("ADMIN", "MANAGER") to an element.

Builds a `<span>` with `Renderer2` and colours it from a palette lookup. An `effect` keeps it in step with the role. Note it creates the badge only once and then updates its text and colour, rather than destroying and rebuilding it on each change.

### shared/directives/click-outside.directive.ts

Attribute.

**Its job:** tell the host when a click landed anywhere else on the page.

```
@HostListener('document:click', ['$event'])
onDocumentClick(event: MouseEvent): void {
  if (target && !this.host.nativeElement.contains(target)) {
    this.appClickOutside.emit();
  }
}
```

It listens on the whole document, then asks "is the thing that was clicked inside me?" using `contains`. If not, it emits. This is what closes the account menu, the notification bell and the row action menus when you click away.

### shared/directives/dialog-close.directive.ts

Attribute. The most subtle file in `shared/`.

**Its job:** mark any button inside a dialog as a close button.

```
constructor(@Host() @Optional() private readonly dialogRef: DialogRef | null) {}
```



| LINE                         | WHAT IT MEANS                                                                                                                                                 |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| @Host()                      | "Only look for a DialogRef in the surrounding component, do not search further up the page." Without it, a button could accidentally close some outer dialog. |
| @Optional()                  | "If there is none, hand me <code>null</code> rather than crashing." So using the directive outside a dialog does nothing instead of breaking the app.         |
| this.dialogRef?.close(false) | The <code>?.</code> is what makes the optional case safe.                                                                                                     |

**The connection to remember:** `@Host()` can see `viewProviders` but not `providers`. That is exactly why `ConfirmDialog` declares `DialogRef` in `viewProviders`. Change it to `providers` and this directive silently stops finding it.

## 6.3 shared/pipes

### shared/pipes/initials.pipe.ts

**Its job:** turn "Mei Tanaka" into "MT" for the round avatar circles.

```
{{ auth.displayName() | initials }}
```

It splits on whitespace, takes the first two words, takes the first letter of each, uppercases them. Returns `'??'` for empty input, so an avatar never renders blank.

### shared/pipes/tenure.pipe.ts

**Its job:** turn a joining date into "2.3 yr" or "7 mo".

**Why a pipe and not a method?** A method written in a template runs on *every* change detection pass — potentially thousands of times. A *pure* pipe runs only when its input actually changes, and caches the result otherwise. For a table of 16 rows this is the difference between 16 calculations and 16 thousand.

## 7. features/ — the screens

### 7.1 Login

**features/auth/login.ts + login.html**

**Its job:** sign in, with three demo accounts to make reviewing easy.

| LINE                                                                                           | WHAT IT MEANS                                                                                                                                                                                                   |
|------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>this.fb.nonNullable.group({...})</code>                                                  | Builds the form. <code>nonNullable</code> means a cleared field returns to its starting value rather than becoming <code>null</code> , which removes a whole category of "cannot read property of null" errors. |
| <code>email: ['admin@acme.io', [Validators.required, Validators.email]]</code>                 | Field format: <code>[startingValue, [rules]]</code> . Two rules here: must be filled, must look like an email.                                                                                                  |
| <code>redirectTo =<br/>signal(route.snapshot.queryParamMap.get('redirectTo')<br/>?? '')</code> | Reads the <code>?redirectTo=</code> the auth guard added, so you return to the page you originally wanted.                                                                                                      |
| <code>showError(name)</code>                                                                   | Returns <code>control.invalid</code> && <code>control.touched</code> . Errors appear only after you have visited a field — you are not told off before you have typed anything.                                 |
| <code>if (this.form.invalid) {<br/>this.form.markAllAsTouched(); return; }</code>              | On submitting an invalid form, mark every field touched so all errors become visible at once, then stop.                                                                                                        |
| <code>this.auth.login(account.name, account.role, remember)</code>                             | The actual sign-in. Everything else in the app reacts to this one signal change.                                                                                                                                |
| <code>[type]="showPassword() ? 'text' : 'password'"</code>                                     | The show/hide password toggle, driven by a signal.                                                                                                                                                              |

### 7.2 Dashboard

**features/dashboard/dashboard.ts + dashboard.html**

**Its job:** the home page — seven stat tiles, a chart, recent joiners, activity, birthdays, announcements, attendance.

| LINE                                                                                      | WHAT IT MEANS                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>store = inject(EmployeeStore) and three more stores</code>                          | The dashboard owns no data of its own. It reads four stores and arranges them. That is why the tiles can never disagree with the employee list.                                                                                   |
| <code>constructor() { void this.announcements.load(); ... }</code>                        | Ask each store to load. The employee store loads itself via its <code>onInit</code> hook, so it is not called here.                                                                                                               |
| <code>const observer: Observer&lt;PresenceSnapshot&gt; = { next, error, complete }</code> | An explicit Observer object rather than a single callback, so all three outcomes are handled by name.                                                                                                                             |
| <code>this.feed.presence().pipe(takeUntil(this.destroy\$)).subscribe(observer)</code>     | Subscribe to the live presence feed, but only until this screen closes.                                                                                                                                                           |
| <code>ngOnDestroy() { this.destroy\$.next(); this.destroy\$.complete(); }</code>          | Firing <code>destroy\$</code> makes <code>takeUntil</code> end the stream, which runs the feed's teardown, which clears the timer. Leaving the dashboard genuinely stops the work — this is the memory-leak prevention in action. |
| <code>activity = computed(() =&gt; ...)</code>                                            | Merges two different sources (leave requests and new joiners) into one list, sorts by date, keeps the newest six.                                                                                                                 |
| <code>*appHasRole==='EMPLOYEE'; else: signedOut"</code>                                   | Signed-in users see a welcome message; guests see a prompt to sign in.                                                                                                                                                            |
| <code>[value]="presence()?.onlineNow ?? '-'"</code>                                       | Shows a dash until the first live value arrives, so the tile is never blank.                                                                                                                                                      |

## 7.3 Employees

**features/employees/employees.routes.ts**

**Its job:** the four URLs of the employee section.

| URL               | SCREEN             | PROTECTION                             |
|-------------------|--------------------|----------------------------------------|
| /employees        | The directory list | signed in                              |
| /employees/new    | Add form           | MANAGER+, and warns on unsaved changes |
| /employees/7      | Profile            | data pre-loaded by the resolver        |
| /employees/7/edit | Edit form          | MANAGER+, and warns on unsaved changes |

**Order matters here too:** 'new' is listed before ':id'. If reversed, the router would read "new" as an id and try to load employee number "new".

## features/employees/employee-list/employee-list.ts

The main working screen: search, five filters, sorting, paging, row menus, delete.

| LINE                                                                                     | WHAT IT MEANS                                                                                                                                                                                                |
|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>private searchInput\$ = new Subject&lt;string&gt;()</code>                         | A stream fed by every keystroke.                                                                                                                                                                             |
| <code>.pipe(debounceTime(250), distinctUntilChanged(), takeUntil(this.destroy\$))</code> | Three operators doing three jobs: wait for a 250ms pause in typing; ignore a term identical to the last; stop entirely when the screen closes. Typing "Engineering" causes one filter operation, not eleven. |
| <code>sorted = computed(() =&gt; [...this.store.filtered()].sort(...))</code>            | Note [...] — sorting a <i>copy</i> , because <code>.sort()</code> rearranges the original array in place, which would corrupt the store's data.                                                              |
| <code>visible = computed(() =&gt; this.sorted().slice(start, start + pageSize))</code>   | One page's worth of rows. Three computeds chained: store filters → component sorts → component pages.                                                                                                        |
| <code>sortBy(key)</code>                                                                 | Clicking the same column again flips the direction; clicking a new column sorts ascending.                                                                                                                   |
| <code>this.page.set(1)</code> after every filter change                                  | Return to page 1. Without it you could be stranded on page 4 of a 1-page result and see nothing.                                                                                                             |
| <code>pendingDelete = signal&lt;Employee   null&gt;(null)</code>                         | Which employee the confirmation dialog is asking about. <code>null</code> means no dialog is open.                                                                                                           |

## features/employees/employee-form/employee-form.ts

One component used for both adding and editing — 20 fields with full validation.

| LINE                                                                                   | WHAT IT MEANS                                                                                                                                                                               |
|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>isEdit = computed(() =&gt; this.employeeId() !== null)</code>                    | Is there an id in the URL? That single question decides the heading, the button text, and whether it creates or updates. One component, two jobs.                                           |
| <code>function adultValidator(control)</code>                                          | A custom validator: rejects a date of birth under 18 years ago. It returns <code>null</code> when valid and <code>{ tooYoung: true }</code> when not — that is the required shape.          |
| <code>function notFutureValidator(control)</code>                                      | Rejects a joining date in the future.                                                                                                                                                       |
| <code>Validators.pattern(/^+[+]?\d\s-]{8,18}\$/)</code>                                | Phone format: an optional +, then 8–18 digits, spaces or dashes.                                                                                                                            |
| <code>hasUnsavedChanges() { return this.form.dirty &amp;&amp; !this.saving(); }</code> | What the leave-warning guard asks. <i>dirty</i> means the user has actually edited something. The <code>!saving</code> stops it warning you while it is in the middle of a successful save. |
| <code>this.form.markAsPristine()</code> before navigating                              | Clears the dirty flag after saving, so a successful save does not trigger the "unsaved changes" warning on the way out.                                                                     |
| <code>skills: value.skills.split(',').map((s) =&gt; s.trim()).filter(Boolean)</code>   | Turns "Figma, CSS, " into <code>['Figma', 'CSS']</code> . <code>filter(Boolean)</code> drops empty strings left by trailing commas.                                                         |
| <code>focusFirstError()</code>                                                         | Finds the first invalid field and moves focus to it, so the error is never off-screen below the fold.                                                                                       |
| <code>toSignal(this.route.paramMap, { requireSync: true })</code>                      | Converts the router's observable into a signal. <code>requireSync</code> promises a value is available immediately, so the code never has to handle "no id yet".                            |

## features/employees/employee-detail/employee-detail.ts

**Its job:** one person's full profile, with five tabs.

| LINE                                                                                                        | WHAT IT MEANS                                                                                                                                              |
|-------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>employee = toSignal(this.route.data.pipe(map((d) =&gt; d['employee'])), { requireSync: true })</code> | Reads what the <i>resolver</i> already fetched. The data is guaranteed present, so the template needs no loading state at all.                             |
| <code>tab = signal&lt;Tab&gt;('personal')</code>                                                            | Which tab is showing. Tab switching is just a signal change.                                                                                               |
| <code>manager = computed(...) / teammates = computed(...)</code>                                            | Found from the store already in memory — no extra network requests to show who someone reports to.                                                         |
| <code>print() { this.showPrint.set(true); setTimeout(() =&gt; globalThis.print?().(), 0); }</code>          | Show the printable block first, then open the print dialog. The <code>setTimeout(..., 0)</code> gives the browser one moment to render it before printing. |

## features/employees/employee-profile/employee-profile.ts

Deliberately written with *both* the old and new Angular APIs, side by side, to show the difference.

| OLD WAY (DECORATOR)                                              | NEW WAY (SIGNAL)                                  |
|------------------------------------------------------------------|---------------------------------------------------|
| <code>@Input({ required: true }) employee!: Employee;</code>     | <code>dense = input(false)</code>                 |
| <code>@Output() edit = new EventEmitter&lt;Employee&gt;()</code> | <code>remove = output&lt;Employee&gt;()</code>    |
| <code>@ViewChild('noteBox') noteBoxRef?</code>                   | <code>headerRef = viewChild(...)</code>           |
| <code>@ViewChildren('skillChip') skillChips?: QueryList</code>   | <code>skillChipSignals = viewChildren(...)</code> |

| LINE                                                      | WHAT IT MEANS                                                                                                                                                                                                              |
|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>showSalary = input(true, { transform: ... })</code> | A transform lets the input accept both a real boolean binding and a plain string attribute ( <code>showSalary="false"</code> ), normalising both.                                                                          |
| <code>onSkillKeydown(event)</code>                        | Arrow-key navigation across skill chips — the reason <code>@ViewChildren</code> is needed, since the handler needs the live list of elements to decide which to focus next. <code>%</code> wraps around at the ends.       |
| <code>ngAfterViewInit()</code>                            | <code>@ViewChild</code> results only exist from this hook onwards — reading them in the constructor gives <code>undefined</code> . Signal queries have no such restriction, which is the main reason they were introduced. |

## features/employees/employee-documents/employee-documents.ts

**Its job:** the documents held against an employee.

The file itself is never uploaded — this portal has no storage backend — but the metadata genuinely round-trips through the API, so the list, the counts and the delete are real. Upload is `*appHasRole="['MANAGER', 'ADMIN']"`; delete is `*appHasRole="'ADMIN'"`.

## 7.4 Departments

### features/departments/departments.routes.ts

**Its job:** show how *nested* routes work.

The list screen is the parent, and the detail screen is its *child*. Visiting `/departments/Engineering` keeps the list on screen and renders the detail panel inside it, because the list template contains its own `<router-outlet />`. Contrast this with employees, where the detail page *replaces* the list entirely.

### features/departments/department-list.ts

**Its job:** list departments with headcounts, plus a create/edit form and delete confirmation.

| LINE                                                               | WHAT IT MEANS                                                                                                                    |
|--------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <code>headcounts = computed(() =&gt; ...Map...)</code>             | Counts employees per department once, into a Map, so each row is an instant lookup rather than a fresh scan of every employee.   |
| <code>Validators.pattern(/^[A-Z]{2,5}\$/)</code>                   | Department codes must be 2–5 capital letters, e.g. <code>ENG</code> .                                                            |
| <code>editing = signal&lt;DepartmentRecord   null&gt;(null)</code> | Whether the form is editing an existing record or creating a new one — one form serving both, exactly as the employee form does. |
| <code>startEdit(department)</code>                                 | Fills the form with the existing values via <code>form.reset({...})</code> and opens it.                                         |

## features/departments/department-detail.ts

**Its job:** everyone in one department.

Reads the department name from the URL with `toSignal(this.route.paramMap)`, then filters the employee store. The payroll total is shown only to managers, guarded by `@if (auth.hasRole('MANAGER'))`.

## 7.5 Attendance and Leave

### features/attendance/attendance-page.ts

**Its job:** who was present, late, absent or on leave on a chosen day.

| LINE                                                                                                                                                                         | WHAT IT MEANS                                                                                                                                                               |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>effect(() =&gt; { this.store.date(); void this.store.load(); })</code>                                                                                                 | Reading <code>date()</code> registers it as a dependency, so changing the date automatically refetches. The read looks pointless but is exactly what makes the effect fire. |
| <code>const scoped =<br/>this.auth.hasRole('MANAGER') ?<br/>this.store.forDate() :<br/>this.store.forDate().filter(r =&gt;<br/>r.employeeId === this.ownEmployeeId())</code> | Managers see everyone; an employee sees only their own rows. The same rule is enforced in the API.                                                                          |
| <code>ownEmployeeId = computed(...)</code>                                                                                                                                   | Matches the signed-in display name against a directory record. A demo shortcut — a real system would carry the employee id inside the session token.                        |

### features/leave/leave-page.ts

**Its job:** apply for leave, and let managers approve or reject.

| LINE                                                                                                   | WHAT IT MEANS                                                                                                                                                                |
|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>requestedDays = computed(() =&gt; countLeaveDays(from, to))</code>                               | Live working-day count as you pick dates, so you see the cost before submitting.                                                                                             |
| <code>if (countLeaveDays(from, to) === 0) { this.form.controls.to.setErrors({ range: true }); }</code> | A cross-field check that cannot be a simple validator: zero working days means the range is backwards or entirely a weekend. The error is attached to the field it concerns. |
| <code>rows = computed(...)</code>                                                                      | Same scoping rule as attendance: employees see their own requests, managers see all.                                                                                         |
| <code>approve()</code> vs <code>rejecting</code> signal                                                | Approving is one click. Rejecting opens a confirmation dialog first — the destructive action gets the extra step.                                                            |
| <code>Validators.minLength(8)</code> on <code>reason</code>                                            | Forces a real explanation rather than "x".                                                                                                                                   |

## 7.6 The admin area

This section is deliberately built the *old* way — with `NgModules` — while the rest of the app is standalone. It shows both styles, and gives the module its own injector scope.

### features/admin/admin.module.ts

| LINE                                                                       | WHAT IT MEANS                                                                                                                                                                                                           |
|----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>@NgModule({ imports: [...] })</code>                                 | The older way of grouping. The components appear under <code>imports</code> rather than <code>declarations</code> because they are themselves standalone.                                                               |
| <code>{ provide: Logger, useFactory: scopedLoggerFactory('admin') }</code> | Inside the admin area, <code>Logger</code> resolves to a logger tagged <code>'admin'</code> instead of the root one. The rest of the app is unaffected. This is <i>hierarchical injection</i> : a closer provider wins. |
| <code>...provideFeatureFlags(...)</code>                                   | Two more flags, <i>added</i> to the root ones because of <code>multi: true</code> . The admin Settings page shows all four merged.                                                                                      |

### features/admin/admin-routing.module.ts

| LINE                                                                 | WHAT IT MEANS                                                                                                                                                            |
|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>RouterModule.forChild(routes)</code>                           | <code>forChild</code> (not <code>forRoot</code> ) because this is a feature area adding routes to an app that already has a router.                                      |
| <code>canActivateChild: [roleGuard('MANAGER', 'ADMIN')]</code>       | Checked on <i>every</i> move between admin sub-pages, not just on entry. So dropping your role mid-session takes effect immediately rather than at the next full reload. |
| <code>{ path: 'settings', canActivate: [roleGuard('ADMIN')] }</code> | One page inside the area demands a higher role than the area itself.                                                                                                     |

### features/admin/pages/admin-shell.ts

**Its job:** the sub-navigation bar and the outlet all admin pages render into.



The shell is created once and stays put while you move between admin pages — only the content inside its `<router-outlet />` is swapped. It injects `Logger` and receives the *admin-scoped* one, proving the module's provider took effect.

**features/admin/pages/admin-overview.ts**

**Its job:** headcount, active count, annual payroll, department bars.

Pure presentation — every number is a `computed` already living in `EmployeeStore`.

**features/admin/pages/admin-announcements.ts**

The most security-relevant screen in the project.

**Its job:** write announcements, and show live what sanitisation will keep.

| LINE                                                                                                                      | WHAT IT MEANS                                                                                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>[innerHTML]="bodyValue()"</code>                                                                                    | Renders the author's HTML. Angular sanitises it automatically here — scripts, <code>onerror=</code> handlers and <code>javascript:</code> URLs are stripped before they reach the page. |
| <code>sanitised = computed(() =&gt;<br/>this.sanitizer.sanitize(SecurityContext.HTML,<br/>this.bodyValue()))</code>       | Runs the sanitiser explicitly so the result can be <i>displayed</i> to the author.                                                                                                      |
| <code>wasModified = computed(() =&gt;<br/>sanitised().replace(/\s+/g, '') !==<br/>bodyValue().replace(/\s+/g, ''))</code> | Compares before and after, ignoring whitespace. If they differ, something was removed, and the author is told so.                                                                       |

**Why this matters:** if a manager could save `<script>steal()</script>` into an announcement, it would run on the computer of every employee who opened the dashboard. That attack is called XSS. Angular blocks it by default — the danger is only when a developer deliberately disables the protection with `bypassSecurityTrustHtml`. This project never does, and that choice is documented in the file's own comment.

**features/admin/pages/admin-system.ts**

**Its job:** the operational view — request count, average latency, cache hits, recent requests.

Every number here comes from the interceptors: the profiling interceptor measured it, the profiling service stored it. Cache hits appear as green rows, errors as red. The "Export payroll" button is wrapped in `*appFeatureFlag="'admin.payroll-export'"`, and since that flag is off, the button is not in the page at all.

**features/admin/pages/admin-audit.ts**

**Its job:** show everything the loggers recorded this session.

It injects `LOG_SINK` — the shared array every logger writes into. Entries from the admin area are tagged `admin`; the rest are tagged `root`. Seeing both scopes in one list is the visible proof that hierarchical injection worked.

## features/admin/pages/admin-settings.ts

**Its job:** show resolved configuration, all merged feature flags, and let an admin preview other roles.

The role-switch buttons call `auth.switchRole(role)`. Because the role is a signal, the entire interface reshapes instantly — nav links disappear, buttons vanish from the DOM, permission-gated panels close. It is the fastest way to see the whole permission system working at once.

## 7.7 Settings and error pages

### features/settings/settings-page.ts

**Its job:** your profile, theme choice, and change password.

| LINE                                                     | WHAT IT MEANS                                                                                                                                                                                                    |
|----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>function passwordsMatch(group)</code>              | A <i>group-level</i> validator: it compares two fields against each other, which a single-field validator cannot do. It is passed as <code>{ validators: passwordsMatch }</code> on the group, not on a control. |
| <code>Validators.minLength(8)</code> on the new password | Stricter than the current-password rule, which only needs 6 — new passwords should be better than old ones.                                                                                                      |

**Honest limitation:** the password change is simulated. This portal has no credential store, so the form validates properly and then does nothing. The comment in the file says so plainly.

### features/misc/forbidden.ts and not-found.ts

Two small screens. `forbidden` (403) is where the role guard sends you, and it tells you which role you currently hold so the problem is obvious. `not-found` (404) is caught by the `'**'` wildcard route.

### features/announcements/announcement-panel.ts

**Its job:** the announcements block on the dashboard.

Takes a `limit` input so a compact surface can show only three. It also offers a `[panel-actions]` projection slot, which the dashboard fills with a "Manage" link — but only for managers, using `*appHasRole` on the projected content itself.

## 8. Testing files

There are 160 tests across 18 files. A test is code that runs your code and checks the result, so that a change which breaks something is caught immediately rather than by a user.

**describe / it / expect** — the three words every test uses. `describe` groups related tests, `it` is one test, `expect` is the actual check:

```
it('returns initials', () => {
  expect(new InitialsPipe().transform('Mei Tanaka')).toBe('MT');
});
```

**TestBed** — Angular's test harness. It builds a miniature version of the application so components can be created and inspected in isolation.

### src/app/testing/test-setup.ts

**Its job:** one function that wires up a realistic test environment.

`configureFeatureTest('MANAGER')` resets the fake database, clears the cache, installs the *real* interceptor chain, and signs in at the requested role. Screens are therefore tested through the same stack the real app uses — so a passing test means the store, the HTTP chain and the role rules all genuinely agree, not just that a mock was called.

### src/app/testing/employee.fixture.ts

**Its job:** one sample employee, in one place.

`makeEmployee({ status: 'EXITED' })` gives the standard record with one field changed. When the model gains a field, you edit this file once instead of every test. Notice its `bioHtml` deliberately contains `` — a real attack string, so the sanitisation tests have something genuine to strip.

## What the test files cover

| FILE                                                | WHAT IT CHECKS                                                                                                                                    |
|-----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>core/di.spec.ts</code>                        | Dependency injection: that <code>useExisting</code> really shares one instance, that multi-providers merge, that optional injection returns null. |
| <code>core/change-detection.spec.ts</code>          | That <code>OnPush</code> and signals update the screen when expected.                                                                             |
| <code>core/guards/guards.spec.ts</code>             | That guards allow, redirect and block correctly per role.                                                                                         |
| <code>core/interceptors/interceptors.spec.ts</code> | That the token is attached, the cache short-circuits, errors are normalised, timings are recorded.                                                |
| <code>core/services/auth.service.spec.ts</code>     | Sign in, sign out, remember-me, role ranking.                                                                                                     |
| <code>core/state/employee.store.spec.ts</code>      | Loading, filtering, creating, updating, removing.                                                                                                 |
| <code>features/**/*.spec.ts</code>                  | Each screen: that it renders, filters, validates and respects roles.                                                                              |
| <code>shared/**/*.spec.ts</code>                    | Pipes, directives and the dialog in isolation.                                                                                                    |

Run them with `npm test`.

## 9. Configuration files

### package.json

**Its job:** list the project's libraries and its command shortcuts.

| COMMAND           | WHAT IT DOES                            |
|-------------------|-----------------------------------------|
| npm start         | Run the app locally with live reload.   |
| npm test          | Run all 160 tests.                      |
| npm run build     | Produce the optimised production files. |
| npm run lint      | Check code style and common mistakes.   |
| npm run format    | Auto-format every file with Prettier.   |
| npm run serve:ssr | Run the built server-rendered version.  |
| npm run start:fr  | Run the app in French.                  |

`dependencies` are libraries the running app needs (Angular, RxJS, Express). `devDependencies` are tools needed only while building and testing (TypeScript, ESLint, Vitest, Playwright). Users never download the second group.

### tsconfig.json

**Its job:** tell TypeScript how strict to be, and set up path shortcuts.

| LINE                                                             | WHAT IT MEANS                                                                                                                                                                                 |
|------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>"paths": { "@core/*": [ "../src/app/core/*"], ... }</code> | Import aliases. You write <code>@core/services/auth.service</code> instead of <code>../../../../../core/services/auth.service</code> . Moving a file no longer breaks a dozen relative paths. |
| <code>noImplicitReturns</code>                                   | Every path through a function must return something. Catches the branch you forgot.                                                                                                           |
| <code>noFallthroughCasesInSwitch</code>                          | Catches a missing <code>break</code> in a switch.                                                                                                                                             |
| <code>strictInjectionParameters</code>                           | Angular refuses to inject something whose type it cannot determine.                                                                                                                           |
| <code>tsconfig.app.json / tsconfig.spec.json</code>              | Two variants: the app build <i>excludes</i> test files, the test build includes only them.                                                                                                    |

### project.json and nx.json

**Its job:** tell Nx how to build, serve, test and lint this project.

| LINE                                                                         | WHAT IT MEANS                                                                                                         |
|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>"budgets": [{ "type": "initial",<br/>"maximumError": "1MB" }]</code>   | Fail the build if the first download grows beyond 1MB. A guard rail against accidentally shipping something enormous. |
| <code>"outputMode": "server", "ssr": { "entry":<br/>"src/server.ts" }</code> | Build the server-rendered version too.                                                                                |
| <code>"i18n": { "locales": { "fr": ... } }</code>                            | Registers French as a second language, using the translation file in <code>src/locale/</code> .                       |
| <code>"cache": true (nx.json)</code>                                         | Nx remembers previous results and skips work whose inputs have not changed, so a repeat build is near-instant.        |

## eslint.config.js, .prettierrc, .editorconfig

Three style tools. **ESLint** finds likely mistakes and enforces conventions. **Prettier** formats code automatically (100-character lines, single quotes) so nobody argues about spacing. **.editorconfig** makes every editor agree on indentation and line endings.

## src/styles.css

**Its job:** the design tokens and shared styling for the whole app.

It defines CSS custom properties — `--bg`, `--surface`, `--text`, `--accent` — once, and everything else refers to them. Dark mode simply redefines those same names, so the entire app switches theme without a single component knowing it happened. Status colours (`--good`, `--danger`) are deliberately *not* themed and are always paired with a text label, so meaning never depends on colour alone.

## src/locale/messages.xlf and messages.fr.xlf

The translation files. Any text marked `i18n="@@some.id"` in a template is extracted into `messages.xlf`; `messages.fr.xlf` holds the French wording. Running `npm run start:fr` builds a completely separate French version of the app.

# 10. Recap tables

## Where does each kind of thing live?

| IF YOU WANT TO...                    | LOOK IN                         |
|--------------------------------------|---------------------------------|
| change what data looks like          | core/models/                    |
| change an API call                   | core/services/                  |
| change what the app remembers        | core/state/                     |
| change something about every request | core/interceptors/              |
| change who may open a page           | core/guards/ and app.routes.ts  |
| change a reusable UI piece           | shared/components/              |
| change one screen                    | features/<name>/                |
| change colours or spacing            | src/styles.css                  |
| change app-wide settings             | core/tokens/app-config.token.ts |

## Signals: the four things you will write

| CODE                | MEANING                                             |
|---------------------|-----------------------------------------------------|
| signal(0)           | A value that announces its own changes.             |
| computed(() => ...) | A value derived from other signals. Read-only.      |
| effect(() => ...)   | Code that re-runs on change. For side effects only. |
| x.asReadonly()      | A read-only view others may watch but not change.   |

## The three places a role is enforced

| WHERE        | WHAT IT STOPS                    | STRENGTH                             |
|--------------|----------------------------------|--------------------------------------|
| Route guard  | Opening the URL at all           | Real, but only for navigation        |
| *appHasRole  | The control existing in the page | Courtesy — good design, not security |
| Mock backend | The action actually happening    | The real lock                        |

## Common mistakes this project deliberately avoids

| MISTAKE                                          | WHAT THIS PROJECT DOES INSTEAD                                                             |
|--------------------------------------------------|--------------------------------------------------------------------------------------------|
| <code>list.push(item)</code> on a signal         | <code>update((l) =&gt; [...l, item])</code> — a new array, so the signal actually changes. |
| <code>array.sort()</code> on stored data         | <code>[...array].sort()</code> — sort a copy, because sort rearranges in place.            |
| Forgetting <code>params = params.set(...)</code> | Reassigns, because <code>HttpParams</code> is immutable.                                   |
| A timer that never stops                         | <code>takeUntil(destroy\$)</code> plus a real teardown function.                           |
| Calling a method from a template                 | A pure pipe or a <code>computed</code> , so it does not run on every check.                |
| Trusting the browser for permissions             | Checking the role again in the data layer.                                                 |
| Touching <code>localStorage</code> during SSR    | Checking <code>isPlatformBrowser</code> first.                                             |
| <code>innerHTML</code> with user text            | <code>textContent</code> via <code>Renderer2</code> , or Angular's sanitiser.              |

## The five-sentence summary

### If you remember nothing else:

1. A **component** is one piece of screen; the app is components inside components.
2. A **signal** is a value that announces its own changes, so the screen redraws itself and no code ever says "now update the display".
3. A **store** holds shared data in one place, so two screens can never disagree.
4. Every HTTP request passes through five **interceptors** that time it, sign it, cache it, catch its errors, and finally answer it.
5. **Roles** are checked three times — at the route, in the page, and in the data layer — and only the last one is a real lock.